

Công nghệ phần mềm

Bài 05: Mô hình hóa hệ thống

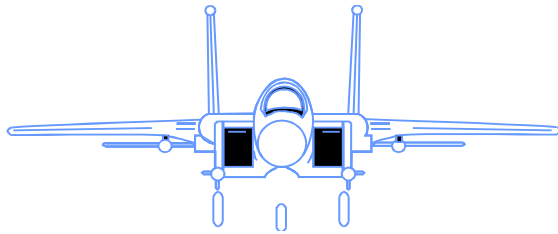
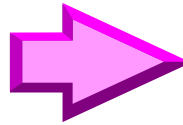
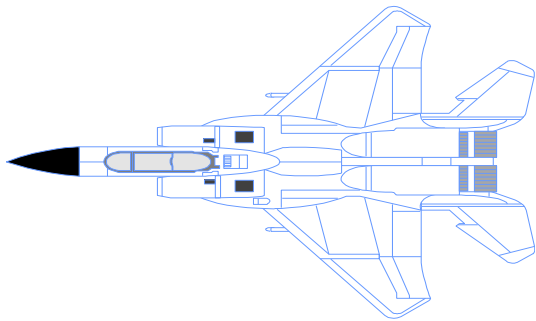
Nội dung

- Khái niệm mô hình hóa hệ thống
- Ngôn ngữ mô hình hóa UML
- Các góc nhìn hệ thống
- Một số loại mô hình hệ thống
 - Mô hình ngữ cảnh
 - Mô hình tương tác
 - Mô hình cấu trúc
 - Mô hình hành vi
- Kỹ nghệ hướng mô hình

Khái niệm cơ bản về hệ thống

- Hệ thống bao gồm các thành phần (thiết bị, phần mềm, con người), hoạt động trong môi trường, và hướng mục đích
- Hệ thống hiện thời (system-as-is): đang vận hành, có vấn đề hoặc nhu cầu cần cải tiến (cơ hội mới)
- Hệ thống được phát triển (system-to-be): cần được phát triển, tiến hóa từ hệ thống hiện thời (giải pháp dựa vào phần mềm – software-to-be).

Mô hình hóa hệ thống



- Mô hình hóa hệ thống: sự đơn giản hóa của chủ thể trong thế giới thực

Mục đích của mô hình hóa hệ thống

- Để hiểu tổng thể hệ thống (chức năng, cấu trúc) và để giao tiếp giữa các bên liên quan
- Mô hình hệ thống hiện thời: giúp cho thu thập và phân tích yêu cầu (kỹ nghệ yêu cầu)
- Mô hình hệ thống mới: (phân tích, thiết kế, cài đặt)
 - đặc tả yêu cầu hệ thống
 - đề xuất thiết kế hệ thống
 - làm tài liệu

Nguyên lý mô hình hóa hệ thống

- Mô hình ảnh hưởng đến cách vấn đề được giải quyết
- Mọi mô hình có thể được biểu diễn ở các mức độ chính xác khác nhau.
- Mô hình tốt nhất là khi được kết nối với thực tế.
- Sử dụng một mô hình duy nhất thường không đủ.

Các góc nhìn hệ thống

- Góc nhìn từ bên ngoài: mô hình hóa ngữ cảnh và môi trường hệ thống
- Góc nhìn tương tác: giữa môi trường và hệ thống, hoặc giữa các thành phần hệ thống
- Góc nhìn cấu trúc: mô hình hóa tổ chức hệ thống, hoặc biểu diễn cấu trúc của dữ liệu hệ thống
- Góc nhìn hành vi: mô hình hóa hành vi động của hệ thống, và cách mà hệ thống phản hồi các sự kiện

Các loại biểu đồ UML

- Biểu đồ hoạt động: biểu diễn các hoạt động diễn ra trong một tiến trình hay một xử lý dữ liệu
- Biểu đồ ca sử dụng: biểu diễn chuỗi tương tác giữa hệ thống và môi trường
- Biểu đồ tuần tự: biểu diễn tương tác giữa tác nhân và hệ thống, hay giữa các thành phần hệ thống
- Biểu đồ lớp: biểu diễn các lớp đối tượng và quan hệ giữa chúng
- Biểu đồ trạng thái: biểu diễn cách hệ thống phản ứng với các sự kiện từ bên trong và bên ngoài

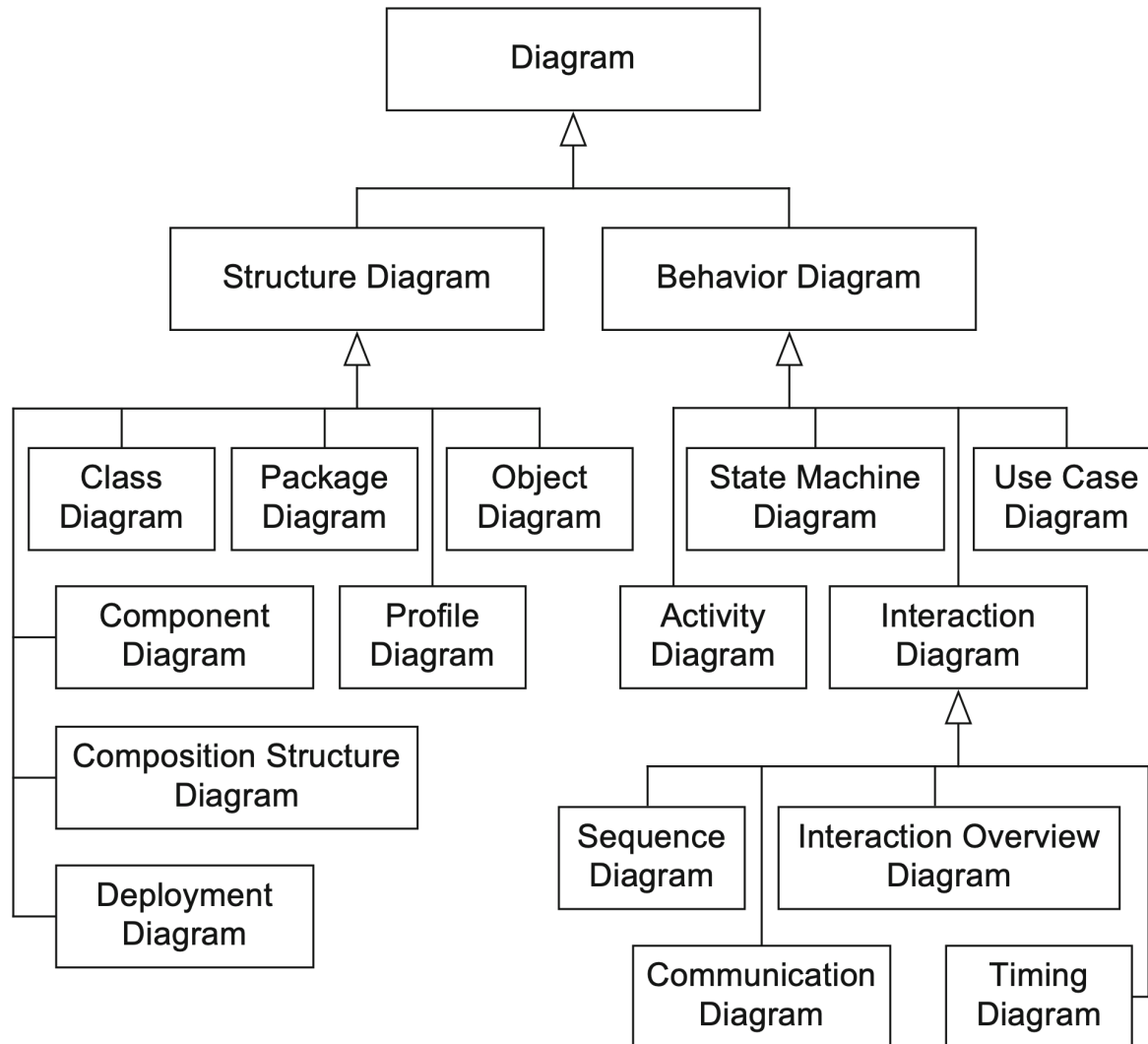
Mô hình ngữ cảnh

- Được dùng để minh họa ngữ cảnh hoạt động của hệ thống (các yếu tố làm nên biên hệ thống)
- Việc xác định biên hệ thống chịu ảnh hưởng của các yếu tố tổ chức và xã hội.
- Là một dạng mô hình kiến trúc chỉ mối quan hệ của hệ thống với các hệ thống khác.

Mô hình quy trình

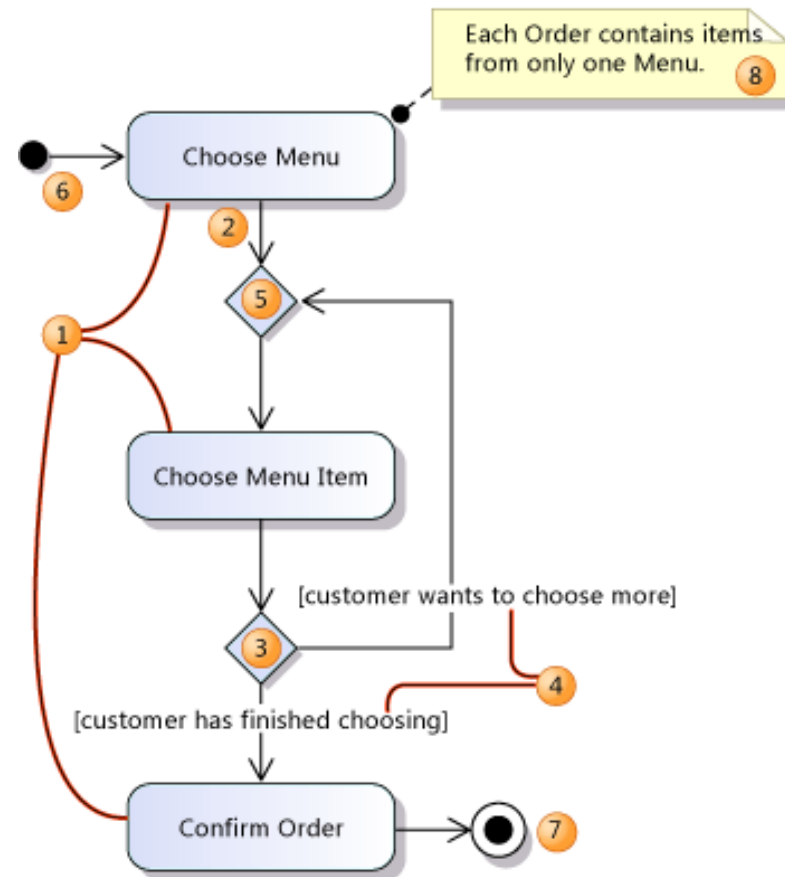
- Thường được dùng để hỗ trợ cho mô hình ngữ cảnh
- Mô tả sự tương tác giữa hệ thống với môi trường trong ngữ cảnh quy trình nghiệp vụ
- Biểu đồ hoạt động UML có thể được dùng để mô hình hóa quy trình nghiệp vụ

Hệ thống các loại biểu đồ



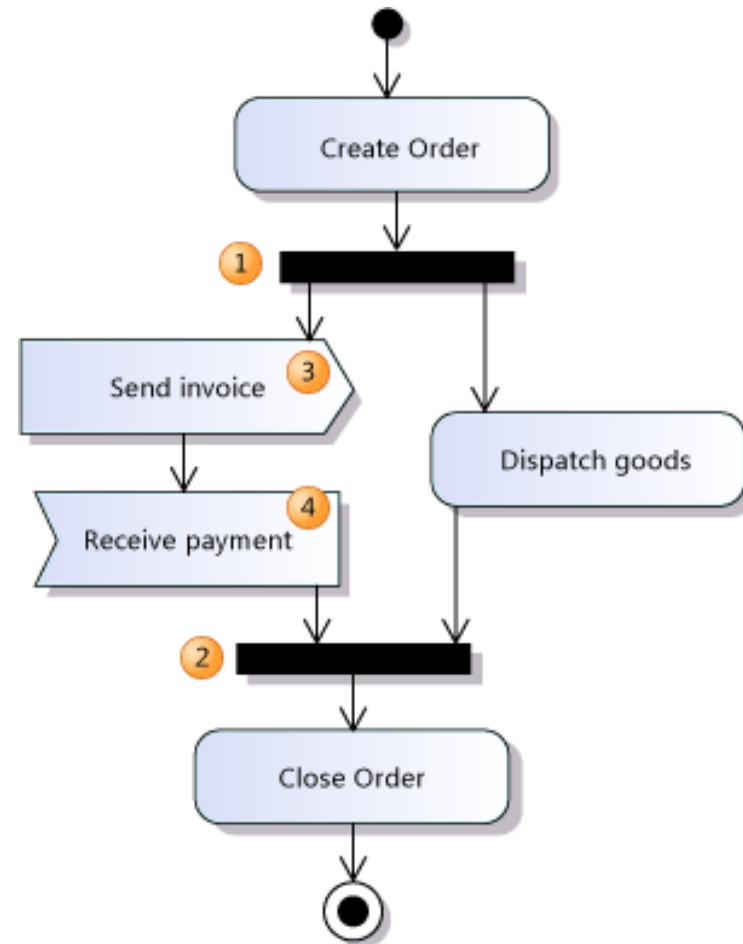
Biểu đồ hoạt động

- Tạo hành động **Action** (1) cho mỗi tác vụ được thực hiện bởi người dùng, hệ thống, hoặc cộng tác cả hai
- Kết nối chuỗi hoạt động bởi các **Connectors** (2)
- Sử dụng **Decision Node** (3) để quyết định bước tiếp theo.
- Đưa vào các điều kiện **guards** (4) để lựa chọn nhánh đi tiếp theo.
- Sử dụng **Merge Node** (5) để hợp nhất nhánh phụ với nhánh chính
- Nút khởi đầu **Initial Node** (6)
- Sử dụng **Activity Final Node** (7) để kết thúc hoạt động



Biểu đồ hoạt động (tiếp)

- **Fork Node** (1) sẽ chia luồng điều khiển thành hai hay nhiều luồng. Khi hành động trước đó kết thúc, tất cả các hành động tương ứng đầu ra của fork node sẽ được kích hoạt.
- **Join Node** (2) sẽ hợp nhất các luồng đồng thời (concurrent threads) với nhau. Hành động sau **Join Node** chỉ được bắt khi tất cả các hành động dẫn đến **Join Node** đều kết thúc.
- Sử dụng **Send Signal Action** (3), và nhận (4), để chỉ rằng một tín hiệu hoặc thông điệp được gửi tới hoặc nhận từ các hoạt động hay tiến trình khác.



Mô hình tương tác

- Mô hình hóa tương tác người dùng giúp xác định các yêu cầu người dùng
- Mô hình hóa tương tác giữa hệ thống và hệ thống giúp làm rõ các vấn đề về giao tiếp (đặc tả giao thức tương tác)
- Mô hình hóa tương tác thành phần giúp hiểu cấu trúc của hệ thống và khả năng đáp ứng các yêu cầu phi chức năng (về hiệu năng và tính tin cậy.)
- Biểu đồ ca sử dụng và biểu đồ tuần tự thường được dùng để mô hình hóa tương tác

Mô hình ca sử dụng

- Ca sử dụng được đề xuất để đặc tả và cấu trúc các yêu cầu hệ thống.
- Mỗi ca sử dụng đại diện cho một tác vụ rời rạc có liên quan đến sự tương tác bên ngoài với một hệ thống.
- Các tác nhân tham gia ca sử dụng có thể là người hoặc các hệ thống khác.
- Mô hình ca sử dụng thường ở dạng kết hợp biểu đồ trực quan và các mô tả văn bản chi tiết.

BIỂU ĐỒ CA SỬ DỤNG

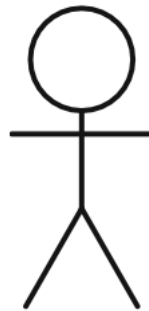
Biểu đồ ca sử dụng

- Biểu diễn chuỗi tương tác giữa hệ thống và môi trường

Tác nhân

- Là một lớp người, tổ chức, thiết bị hoặc thành phần phần mềm bên ngoài tương tác với hệ thống
- Biểu diễn bằng hình người với hai tay giơ ngang

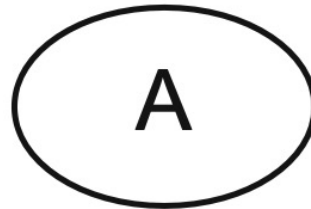
Actor



Ca sử dụng

- Đại diện cho các hành động được thực hiện bởi một hoặc nhiều tác nhân trong việc theo đuổi một mục tiêu cụ thể
- Trong biểu đồ ca sử dụng, các ca sử dụng được kết gán với các tác nhân tham gia ca sử dụng
- Biểu diễn bằng hình elip, nội dung viết theo cú pháp **Verb + Noun**

Use case



Hệ thống

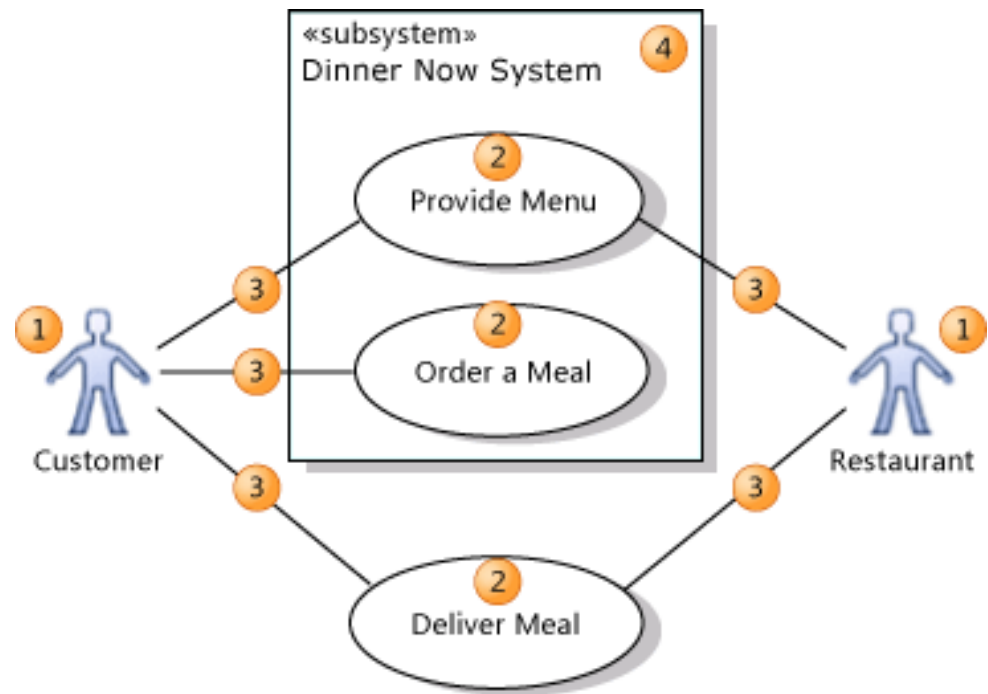
- Là một thành phần phần mềm nhỏ, mà các tác nhân của nó chỉ là các thành phần phần mềm khác; hoặc nó có thể là một ứng dụng hoàn chỉnh.

Biên hệ thống

- Biên hệ thống xác định những gì thuộc về hệ thống và những gì thuộc về môi trường
- Biên hệ thống ảnh hưởng sâu sắc đến yêu cầu của hệ thống
- Xác định biên hệ thống mang yếu tố chính trị
- Áp lực để xác định hệ thống, qua đó làm tăng / giảm ảnh hưởng hoặc khối lượng công việc của các thành phần trong tổ chức.

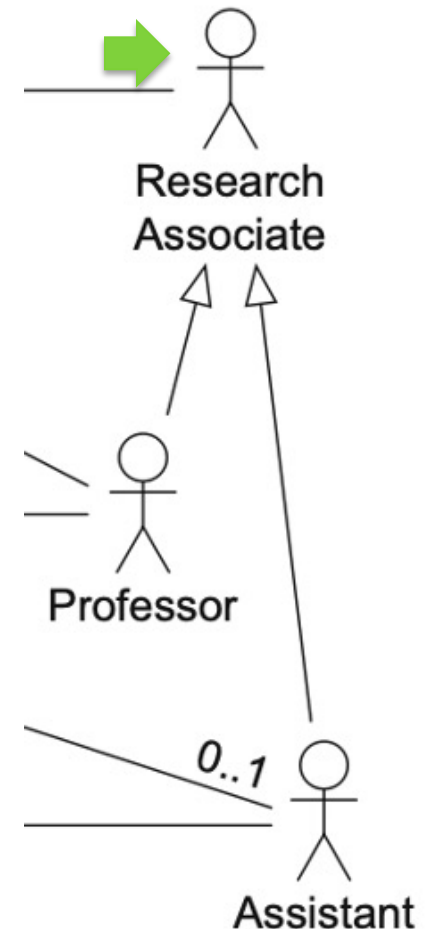
Hệ thống đặt món nhà hàng

- Các ca sử dụng ở đây bao gồm là **Đặt bữa ăn (Order Meal)**, **Cập nhật thực đơn (Update Menu)**, **Thanh toán (Process Payment)**
- Tác nhân: **Khách hàng và Nhà hàng**



Quan hệ tổng quát hóa giữa tác nhân

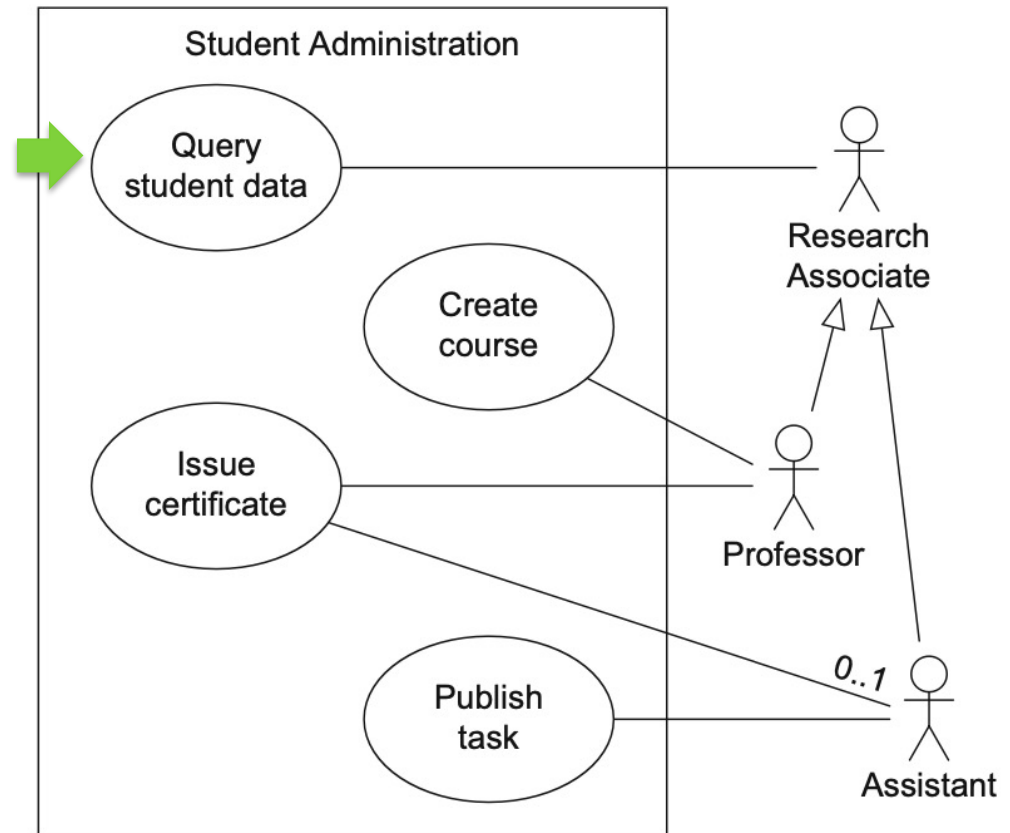
- Giữa các tác nhân có những hành vi chung -> tổng quát hóa (thừa kế)
- Ví dụ: Xây dựng hệ thống giảng dạy
 - Giáo sư (**Professor**) và Trợ lý (**Assistant**) bản chất đều là chuyên viên nghiên cứu (**Research Associate**)
 - Research Associate** sẽ chứa những điểm chung của **Professor** và **Assistant**



Ví dụ quan hệ tổng quát hóa giữa tác nhân

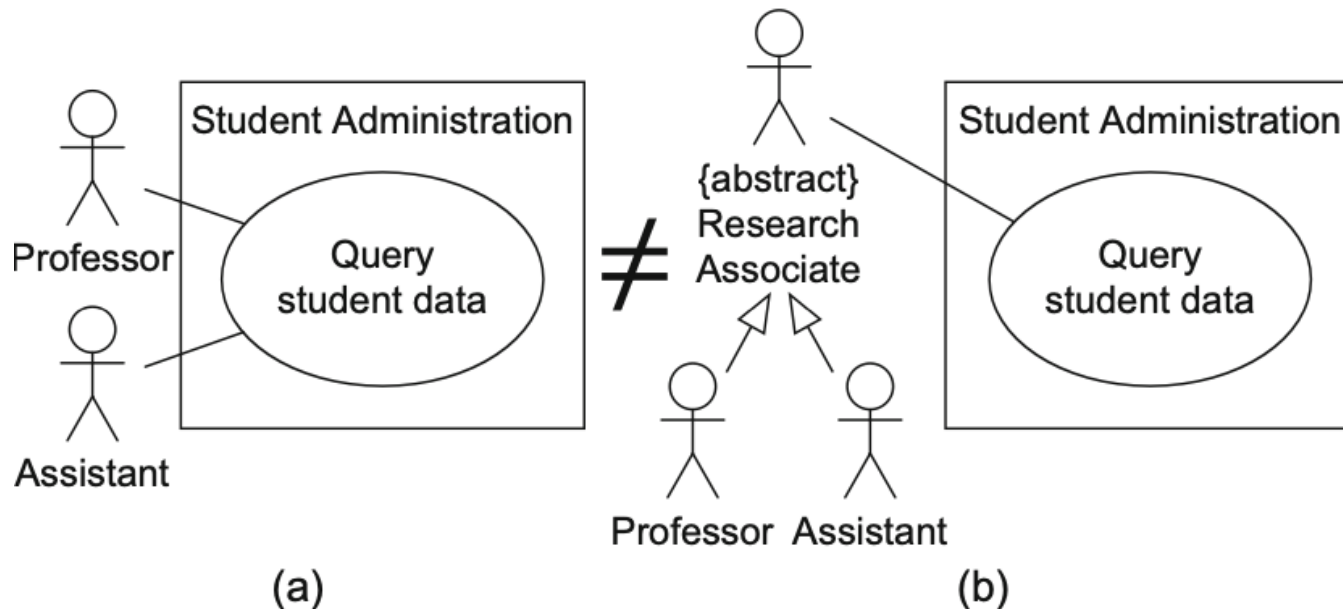
**Professor thừa kế
Research Associate**

-> **Professor** có ca sử dụng **Query student data**



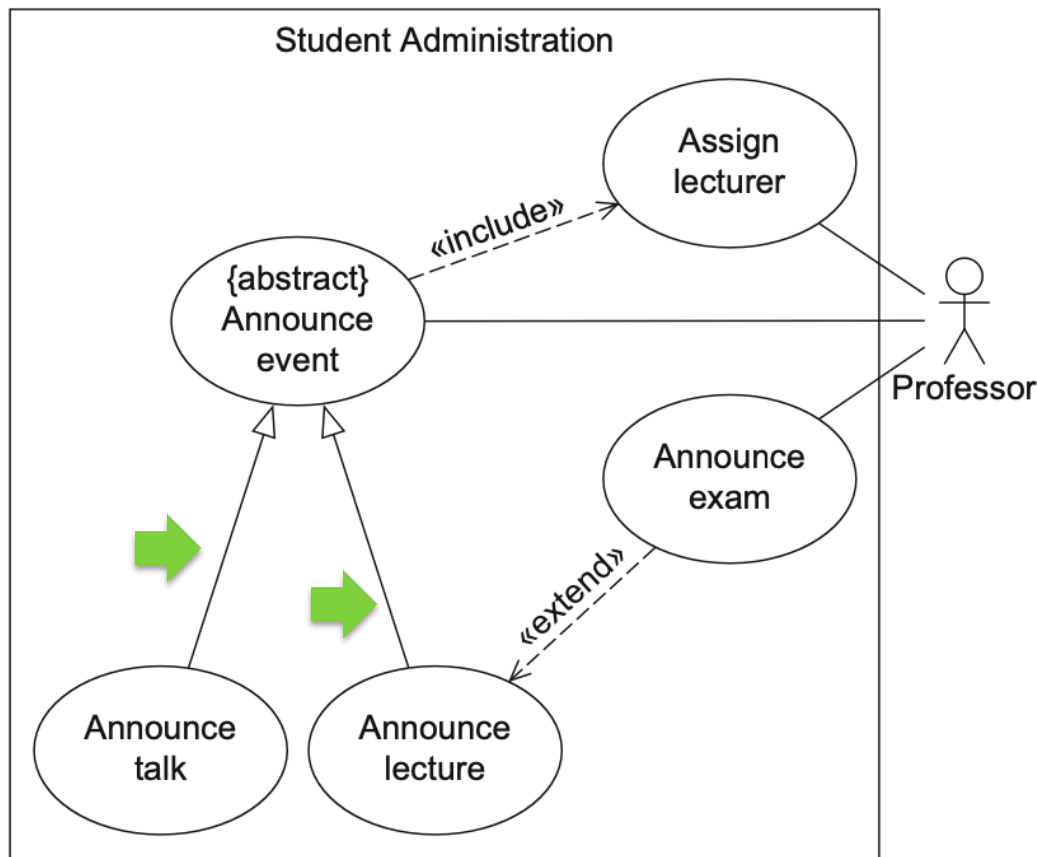
Ví dụ quan hệ tổng quát hóa giữa tác nhân

So sánh khi **không** dùng quan hệ tổng quát hóa (a) và khi dùng (b)

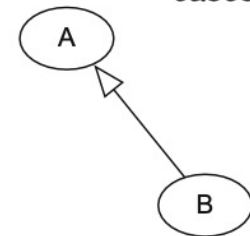


Quan hệ tổng quát hóa giữa các ca sử dụng

- Hành vi chung giữa các ca sử dụng con (B) gộp trong ca sử dụng cha (A)
- Ca sử dụng cha được dùng từ khóa {abstract}

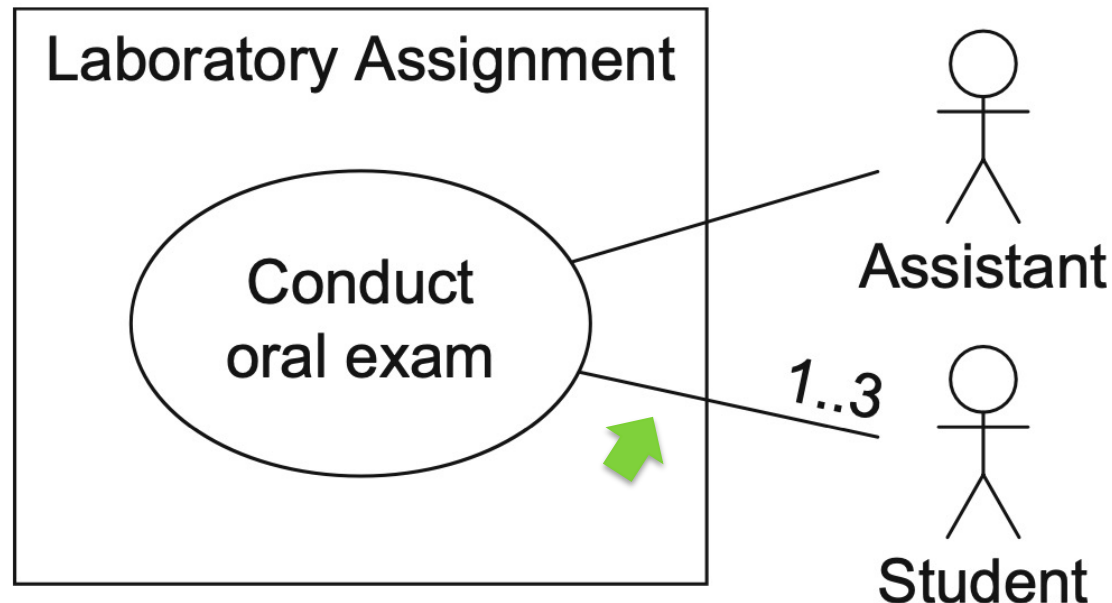


Generalization for use cases



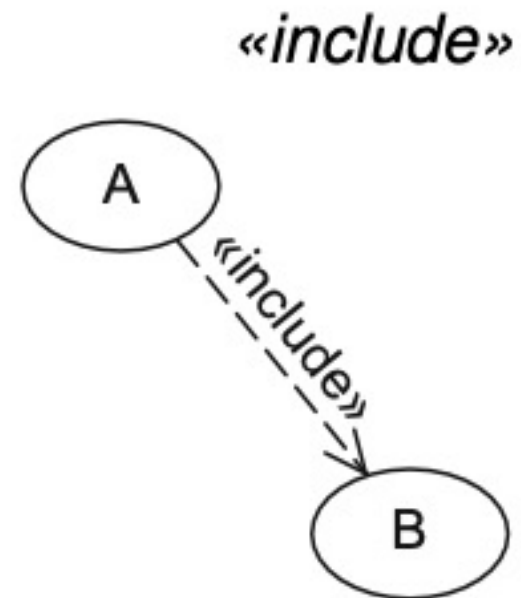
Ràng buộc lực lượng giữa tác nhân và ca sử dụng

- Ràng buộc lực lượng giữa tác nhân và ca sử dụng có thể là 1, 1..*, 0..1, hoặc *.
- Ví dụ: Trong một buổi thi vấn đáp, sẽ có 1->3 sinh viên tham dự



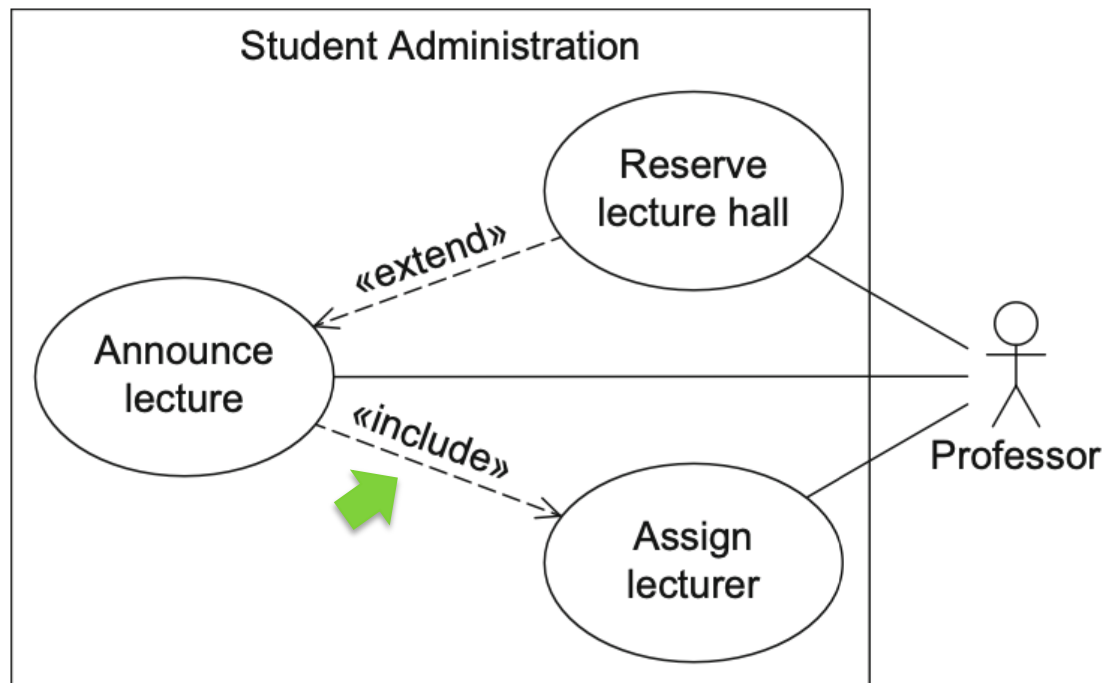
Quan hệ include giữa các ca sử dụng

- Một ca sử dụng diễn đạt một vài chi tiết của ca sử dụng khác
- Mỗi tên nét đứt có đầu nhọn với diễn giải <<include>>
- Ca sử dụng A có thể có nhiều bước, trong đó có một bước là ca sử dụng B



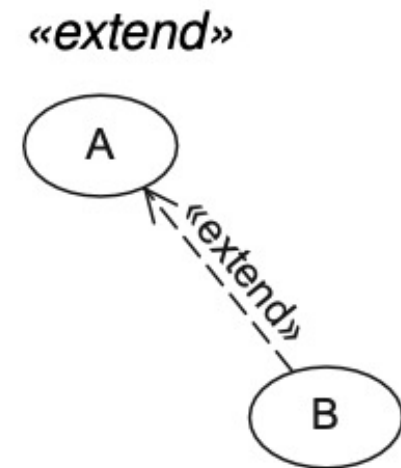
Quan hệ include giữa các ca sử dụng

- Ví dụ: Để thực hiện được ca sử dụng thông báo bài học (**announce lecture**), có một bước phải hoàn thành là chọn giảng viên dạy (**assign lecturer**)



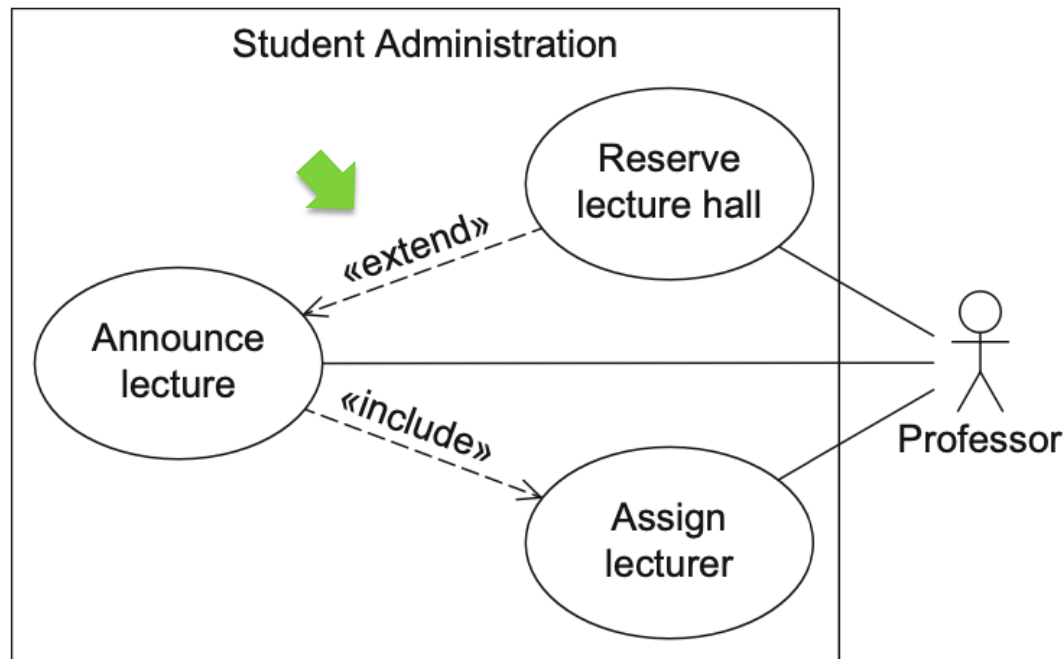
Quan hệ extend giữa các ca sử dụng

- Khi thực hiện ca sử dụng A, dưới một vài điều kiện nào đó, người dùng có thể thực hiện ca sử dụng B
- Mũi tên nét đứt có đầu nhọn với diễn giải <<extend>>



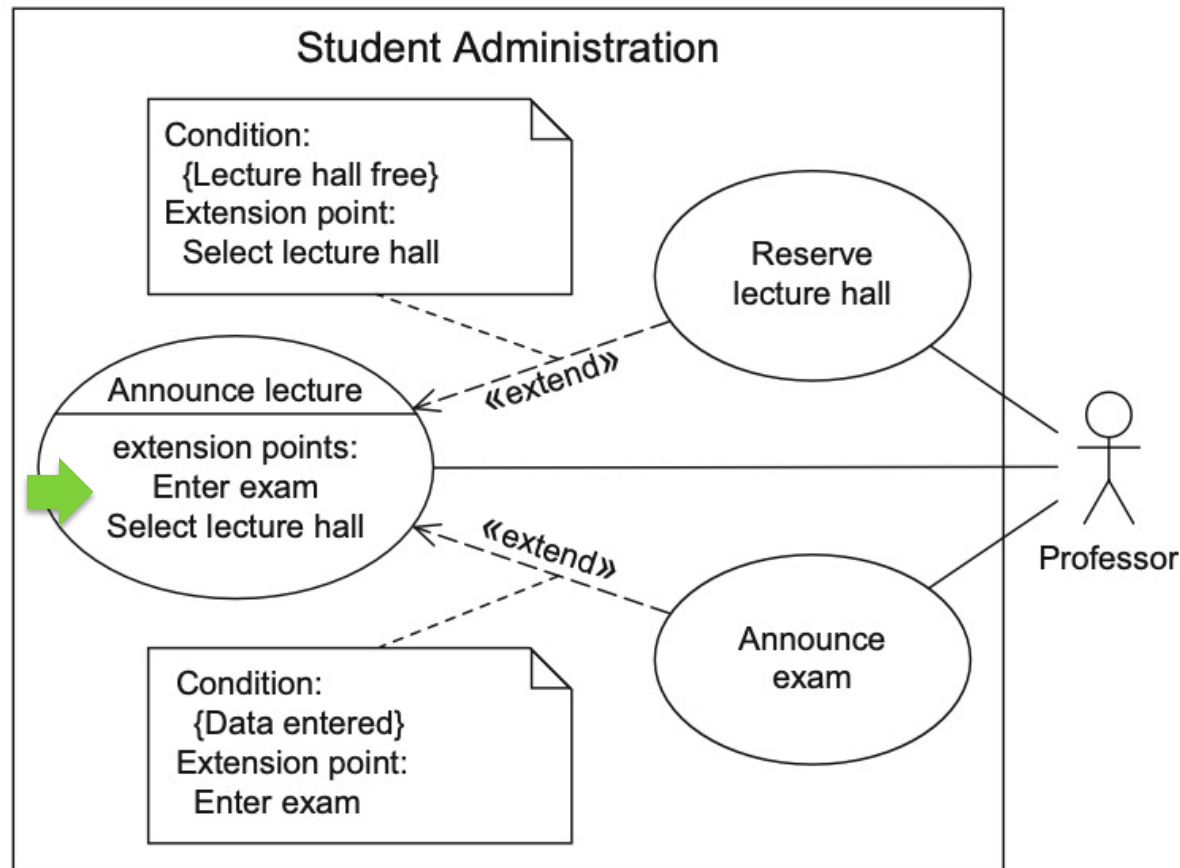
Quan hệ extend giữa các ca sử dụng

- Ví dụ: **Professor** khi thông báo bài giảng (**Announce lecture** - A), ông ta có thể đặt luôn phòng học (**Reserve lecture hall** - B)
- Giao diện khi thực hiện A sẽ có tùy chọn để thực hiện B



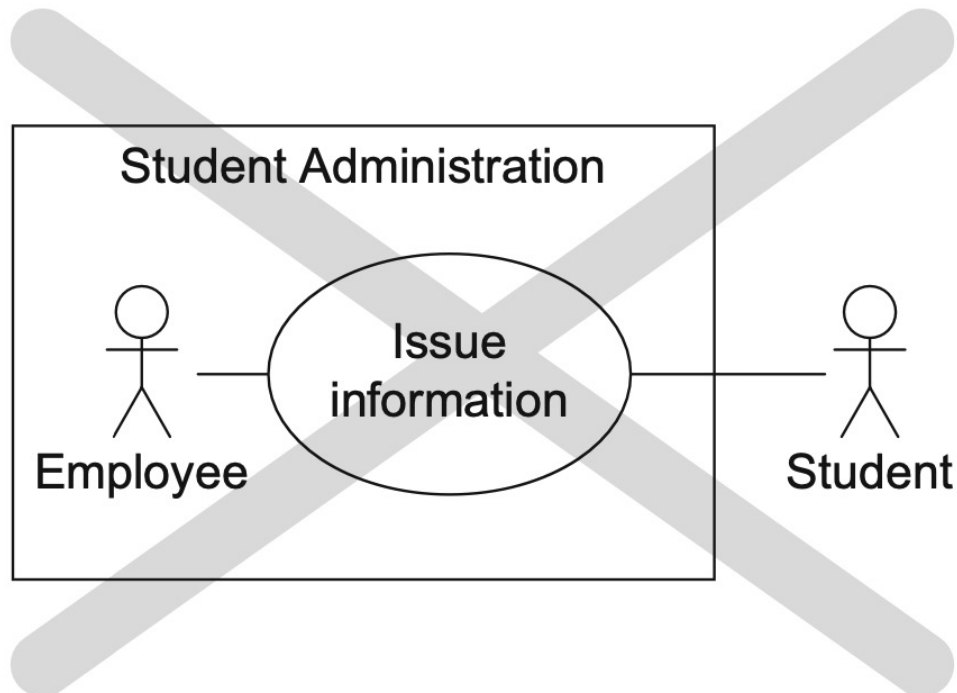
Quan hệ extend giữa các ca sử dụng

- Sử dụng extension point để mô tả điều kiện thực hiện ca sử dụng mở rộng



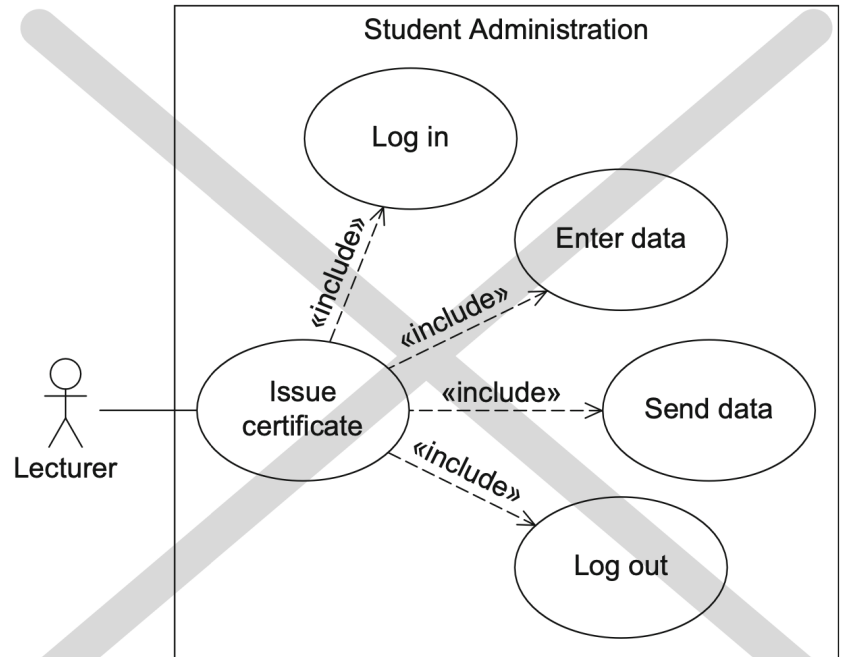
Lỗi phổ biến

- Xác định sai biên hệ thống
- VD: Tác nhân **Employee** không được nằm trong hệ thống!

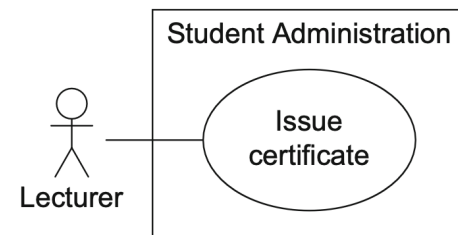


Lỗi phổ biến

- Lỗi phân tách ca sử dụng
- VD: Ca sử dụng **Issue certificate** được chia thành các ca sử dụng con, nhưng **Enter data** không nên là một ca sử dụng
- **Log in** là tiền điều kiện để thực hiện **Issue certificate** -> không nên dùng <<include>>



(a)

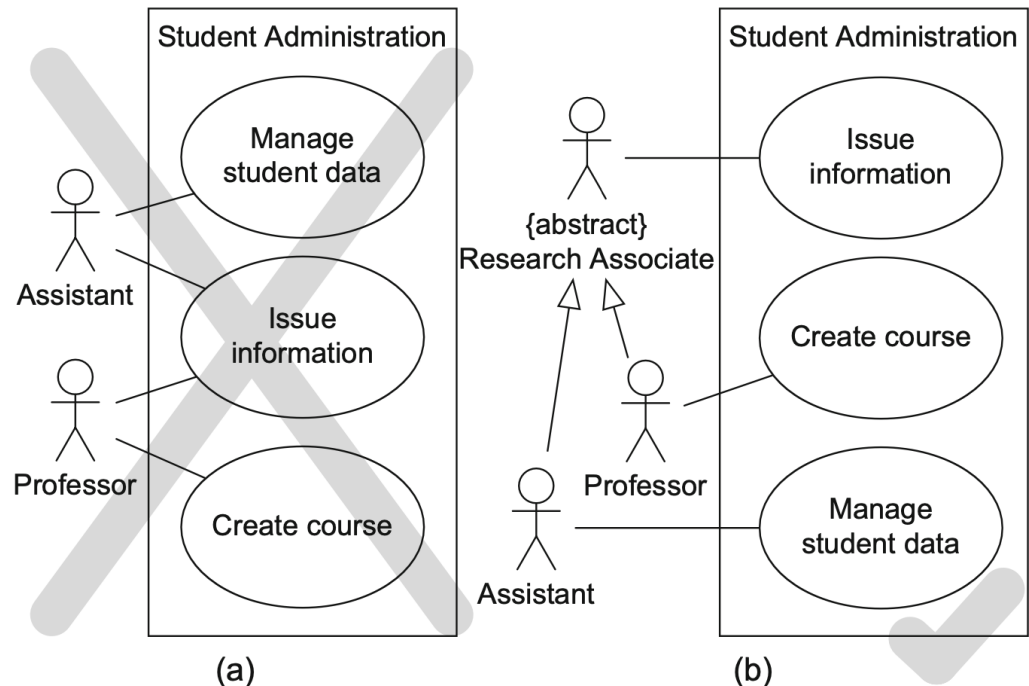


(b)



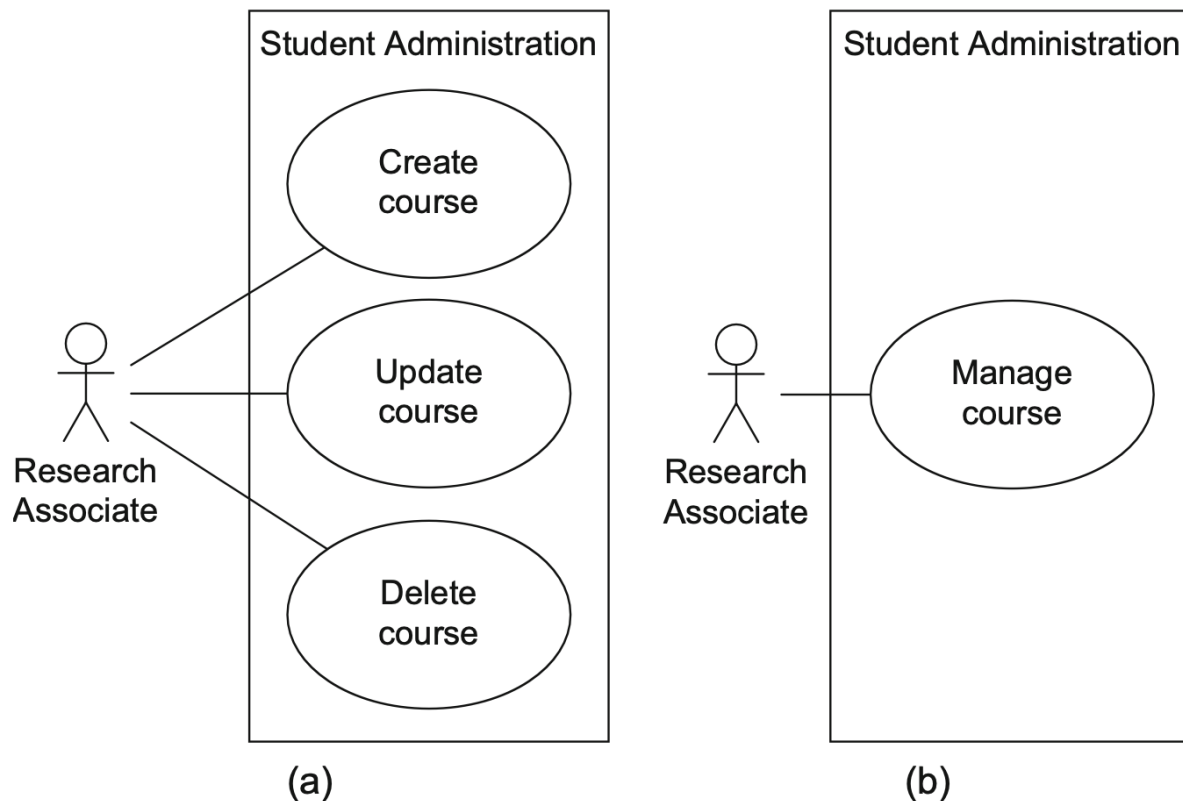
Lỗi phổ biến

- Lỗi sử dụng quan hệ phụ thuộc
- VD: Nếu để **Assistant** và **Professor** cùng thực hiện **Issue Information** -> dễ hiểu nhầm cần hai tác nhân này mới thực hiện được
- Sửa lại: Dùng quan hệ thừa kế tác nhân (hình b)



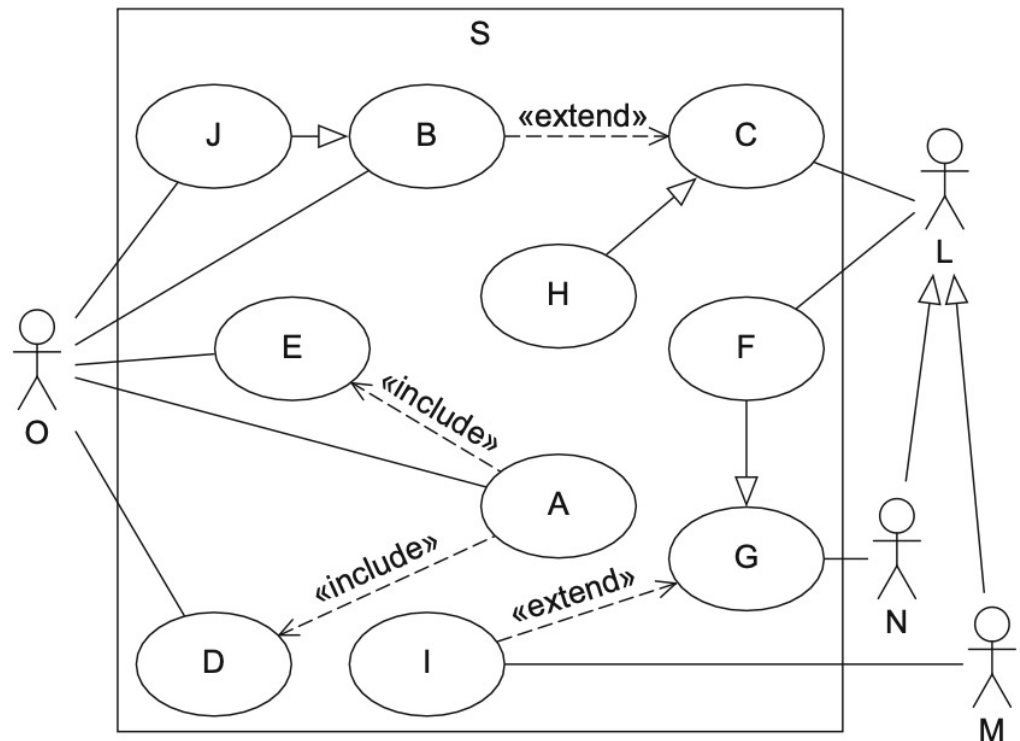
Lỗi phổ biến

- Nên hợp nhất các ca sử dụng cho cùng đối tượng
- VD: Hợp nhất Tạo/Sửa/Xóa course -> **Manage course**
- Giảm thiểu số ca sử dụng -> hiệu quả quản lý biểu đồ ca sử dụng

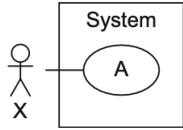
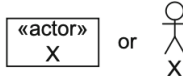
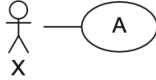
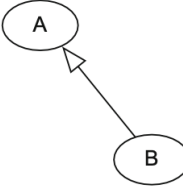
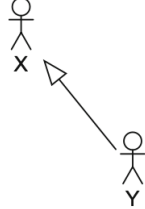
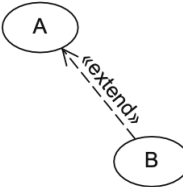
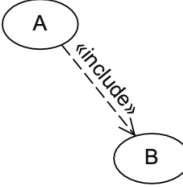


Tóm tắt biểu đồ ca sử dụng

- N có thực hiện được C không? => Có. L thực hiện được C. N thừa kế L => các ca sử dụng của L thì N thực hiện được



Tóm tắt biểu đồ casử dụng

Name	Notation	Description
System		Boundaries between the system and the users of the system
Use case		Unit of functionality of the system
Actor		Role of the users of the system
Association		X participates in the execution of A
Generalization (use case)		B inherits all properties and the entire behavior of A
Generalization (actor)		Y inherits from X; Y participates in all use cases in which X participates
Extend relationship		B extends A: optional incorporation of use case B into use case A
Include relationship		A includes B: required incorporation of use case B into use case A

BIỂU ĐỒ LỚP

Mô hình cấu trúc

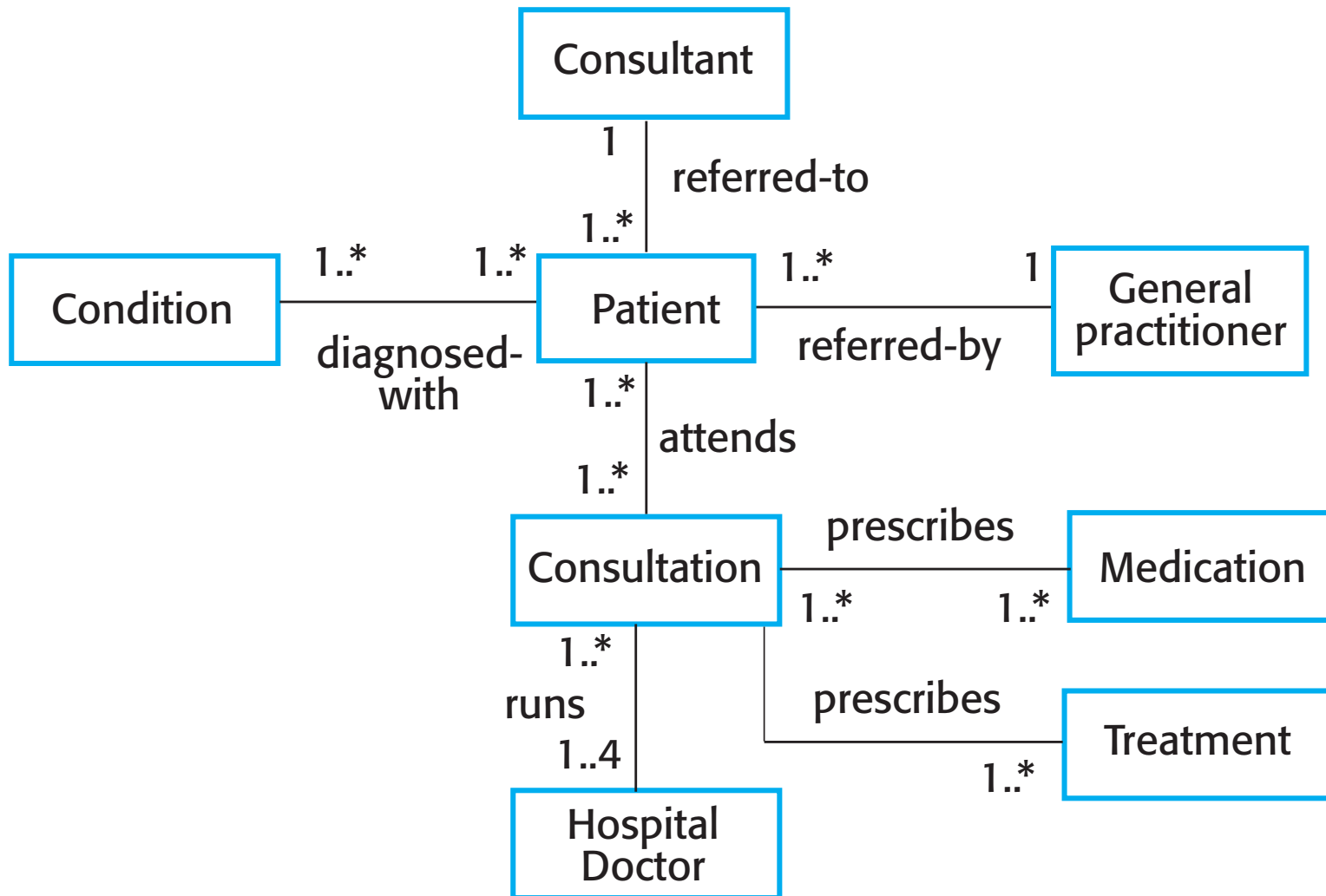
- Được dùng để biểu diễn cách tổ chức của hệ thống, chỉ ra quan hệ giữa các thành phần hệ thống
- Mô hình cấu trúc ở dạng mô hình tĩnh khi mô tả cấu trúc của thiết kế hệ thống hoặc ở dạng mô hình động khi phản ánh cấu trúc hệ thống khi thực thi
- Mô hình này thường được dùng khi thảo luận và thiết kế kiến trúc hệ thống

Biểu đồ lớp

- Được dùng để biểu diễn mô hình cấu trúc hệ thống
- Lớp: tập các đối tượng (thể hiện) cùng đặc điểm, hành vi và ngữ nghĩa
- Quan hệ kết hợp: quan hệ ngữ nghĩa giữa các lớp
- Tổng quát hóa: Lớp con chi tiết hóa lớp cha
- Quan hệ tụ hợp (hợp thành): quan hệ giữa tổng thể và bộ phận, có (không có) thành phần chia sẻ.

Consultation
Doctors Date Time Clinic Reason Medication prescribed Treatment prescribed Voice notes Transcript ...
New () Prescribe () RecordNotes () Transcribe () ...

Ví dụ biểu đồ lớp



Mô hình hành vi

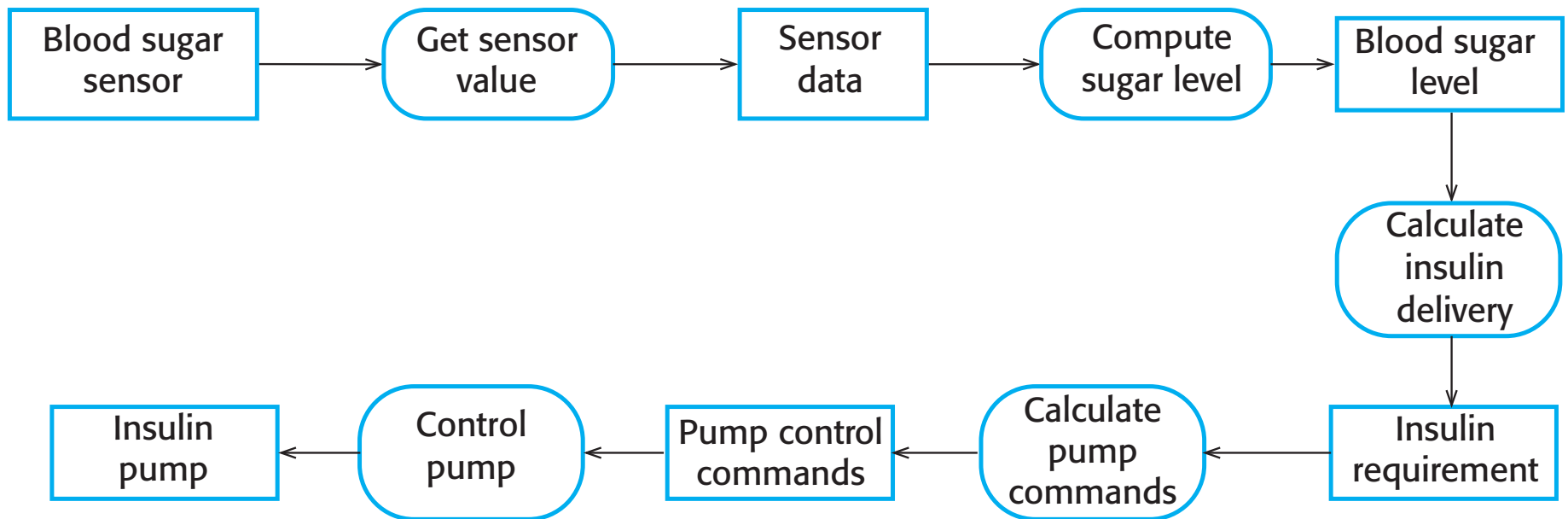
- Mô hình hành vi mô tả hành vi động của hệ thống khi nó thực hiện, qua đó làm rõ cách thức hệ thống phản hồi các kích thích từ môi trường
- Kích thích từ môi trường có 2 dạng:
 - Dữ liệu: được gửi đến và được hệ thống xử lý
 - Sự kiện: kích hoạt các xử lý của hệ thống

Mô hình hóa hướng dữ liệu

- Thường dùng cho các hệ thống nghiệp vụ (business system)
- Mô hình hướng dữ liệu (data-driven models) biểu diễn chuỗi hoạt động xử lý các dữ liệu đầu vào và kết xuất dữ liệu đầu ra tương ứng.
- Được hỗ trợ bởi biểu đồ hoạt động UML 2.0 hoặc biểu đồ tuần tự (ở dạng không tường minh)

Ví dụ mô hình hóa hướng dữ liệu

Biểu đồ luồng dữ liệu (DFD) cho thao tác tiêm Insulin



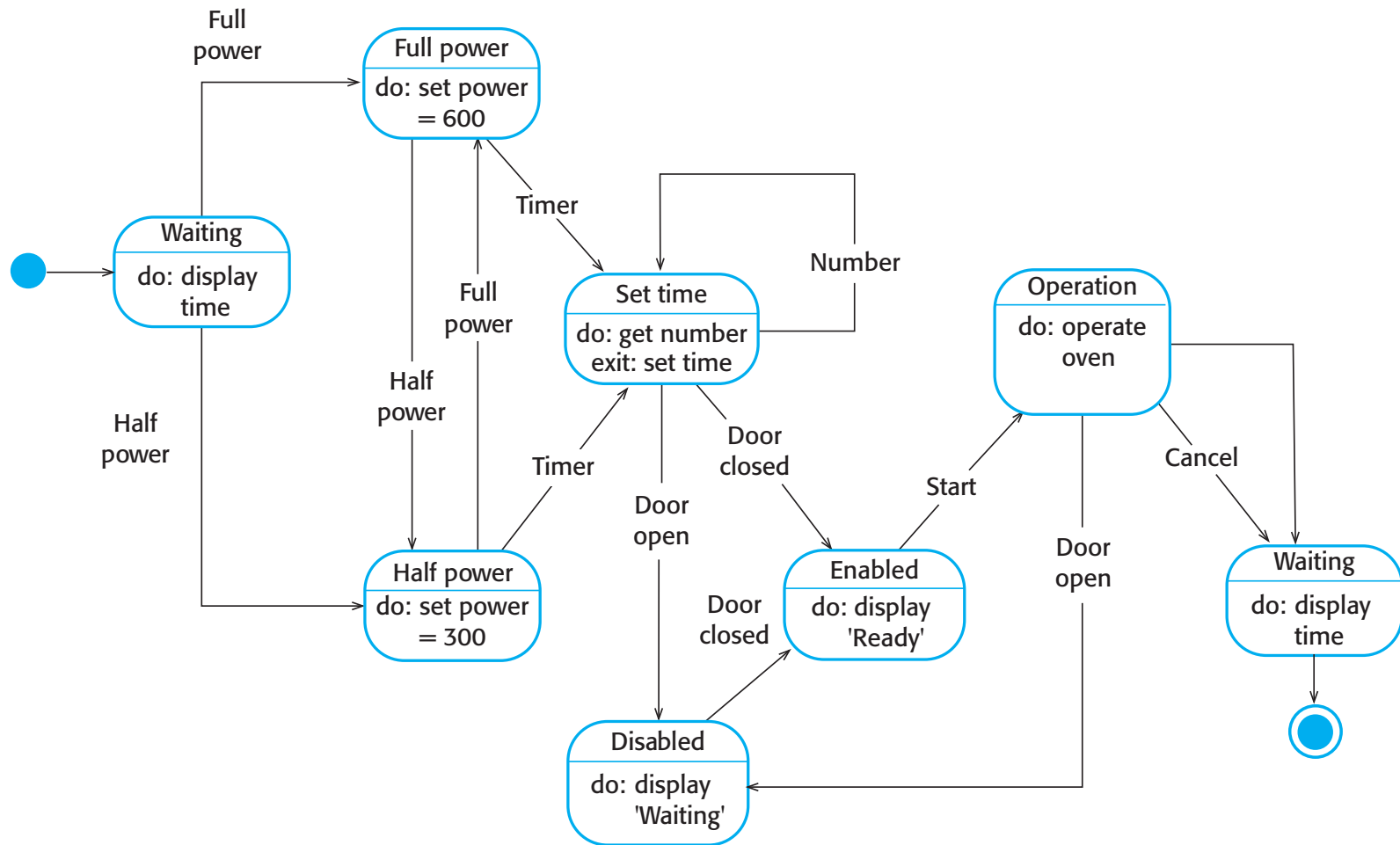
Mô hình hóa hướng sự kiện

- Mô hình hướng sự kiện (event-driven models) biểu diễn cách mà hệ thống phản hồi các sự kiện từ bên ngoài và bên trong.
- Dựa vào giả định cơ bản: Hệ thống có một tập hữu hạn trạng thái và các sự kiện sẽ kích hoạt các chuyển từ trạng thái này sang trạng thái kế tiếp.
- Thường dùng cho các hệ thống thời gian thực (real-time system)

Mô hình máy trạng thái

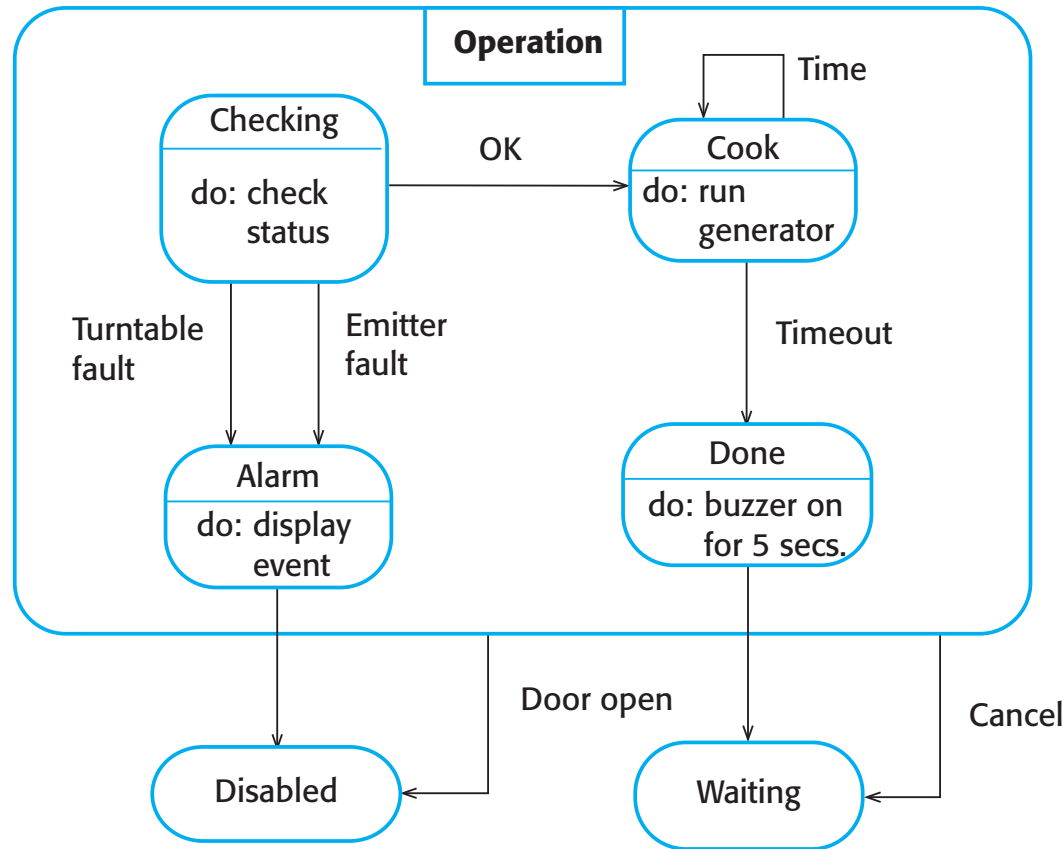
- Được dùng để mô hình hóa hành vi hệ thống. Biểu diễn cách mà hệ thống phản hồi các sự kiện từ bên ngoài và bên trong.
- Mô hình máy trạng thái (trạng thái hệ thống, các sự kiện, các chuyển)
- Biểu đồ trạng thái UML (statecharts) được dùng để biểu diễn các mô hình máy trạng thái.

Ví dụ mô hình máy trạng thái



Mô hình hoạt động của lò vi sóng

Ví dụ mô hình máy trạng thái (2)



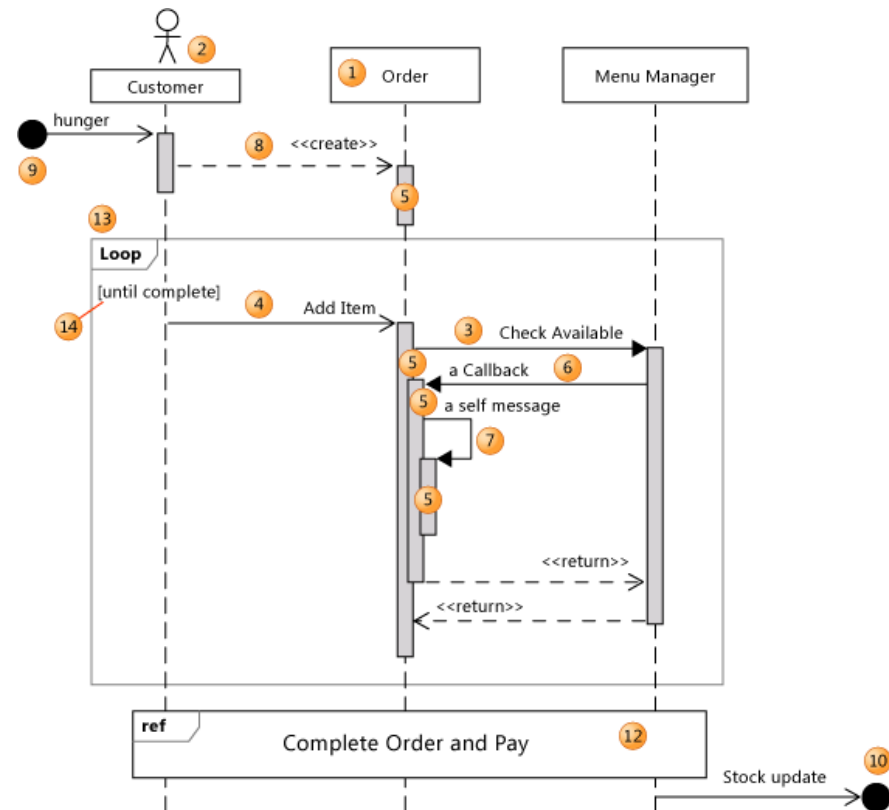
BIỂU ĐỒ TUẦN TỰ

Biểu đồ tuần tự

- Để mô hình hóa tương tác giữa tác nhân và các đối tượng bên trong hệ thống
- Để đặc tả các kịch bản sử dụng

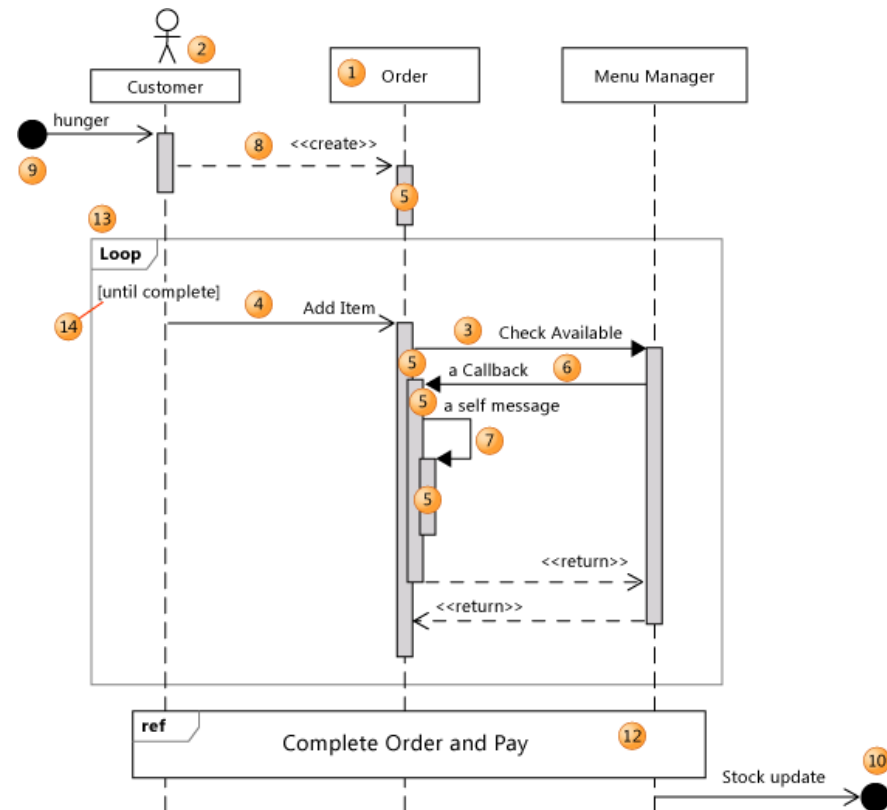
Biểu đồ tuần tự

- Đường sống (**Lifelines**) (1) biểu diễn các thể hiện của lớp, thành phần, tác nhân hay các thiết bị
- Các thông điệp (3, 4, 6, 7) được gửi từ đối tượng gửi đến đối tượng nhận
- Một khoảng thực thi (5) xuất hiện ở đối tượng nhận thông điệp
- Ký pháp (9) để chỉ tình huống một thông điệp được gửi từ nguồn không xác định, và được gửi một cách không đồng bộ



Biểu đồ tuần tự

- Thông điệp đồng bộ (**Synchronous**) (3) mô tả tương tác trong đó đối tượng gửi chờ cho đến khi nhận được phản hồi của đối tượng nhận
- Mũi tên **<<return>>** là tín hiệu từ đối tượng nhận, để kết thúc một thực thi
- Mũi tên bất đồng bộ (**Asynchronous**) (4) để chỉ người gửi có thể tiếp tục công việc mà không phải chờ phản hồi từ người nhận
- Tín hiệu **<<Create>>** (8) để mô tả hành động đối tượng gửi tạo đối tượng nhận



Đường sông (Lifeline)

- Nhiều cách viết tên
 - Dùng vai trò mô tả (a)
 - Dùng lớp để mô tả (b) – tên được đặt sau dấu “:”
 - Kết hợp a và b với cú pháp <vai trò>:<tên lớp>

lecturer

(a) Role

:Professor

(b) Class

lecturer
:Professor

(c) Role/Class

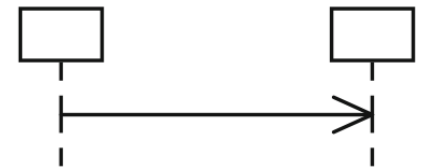
Gửi thông điệp (Message)

- Ba loại thông điệp để gửi
 - Thông điệp đồng bộ: bên gửi **phải đợi phản hồi** mới thực hiện tiếp
 - Thông điệp bất đồng bộ
 - Thông điệp phản hồi
- Chú ý về các dùng kí hiệu mũi tên

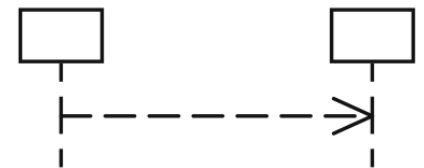
Synchronous message



Asynchronous message

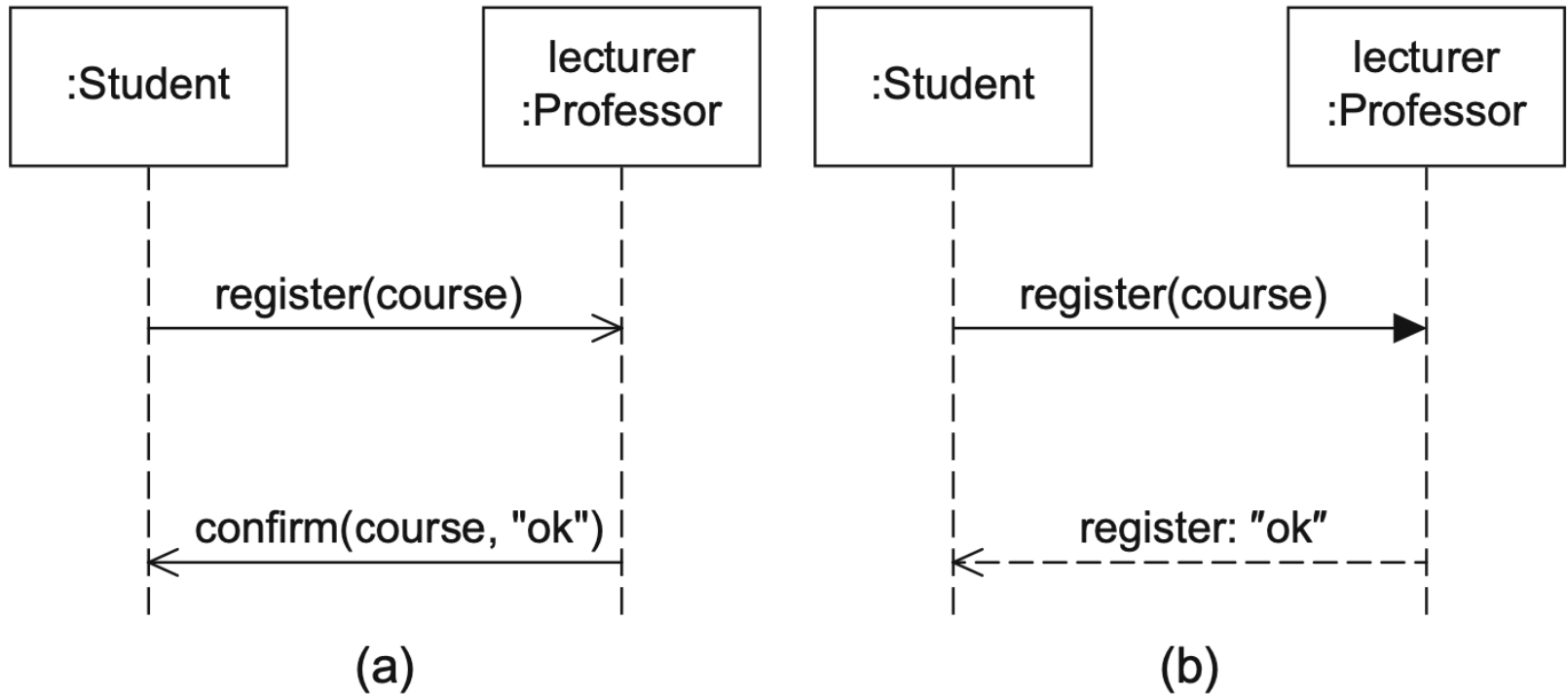


Response message



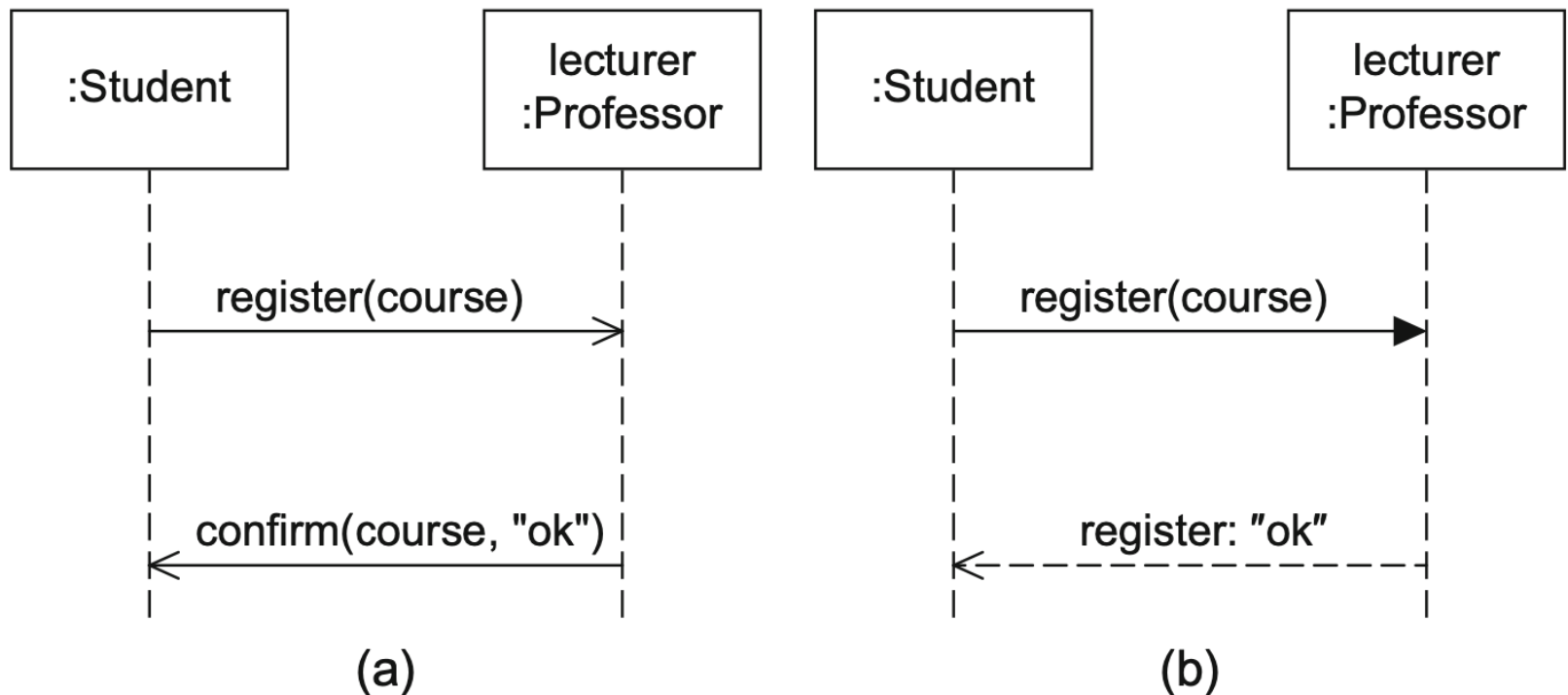
Gửi thông điệp (Message)

- Ví dụ: Hãy phân biệt sự khác nhau giữa a và b



Gửi thông điệp (Message)

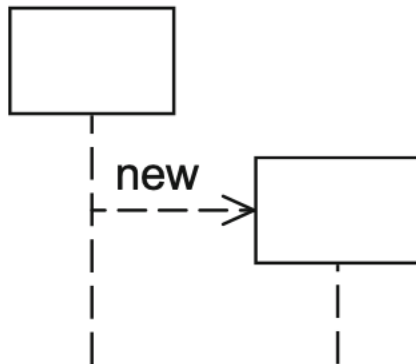
- Hình a: **Student** không cần đợi **Professor** (gọi là **lecturer**) phản hồi (**confirm**) để thực hiện tiếp
- Hình b: **Student** phải đợi **Professor** phản hồi mới thực hiện tiếp được hành động kế tiếp



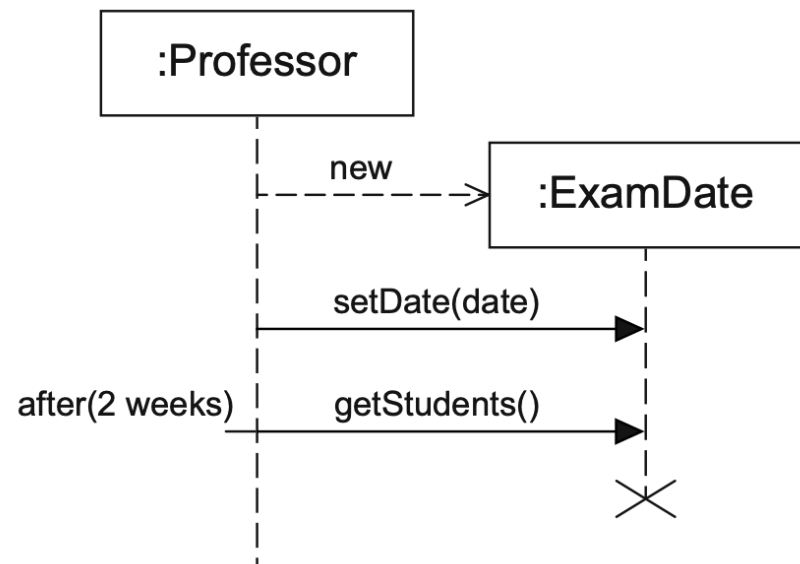
Thông điệp tạo (Create message)

- Ta cần tạo đối tượng
- Dùng từ khóa **new**

Create message



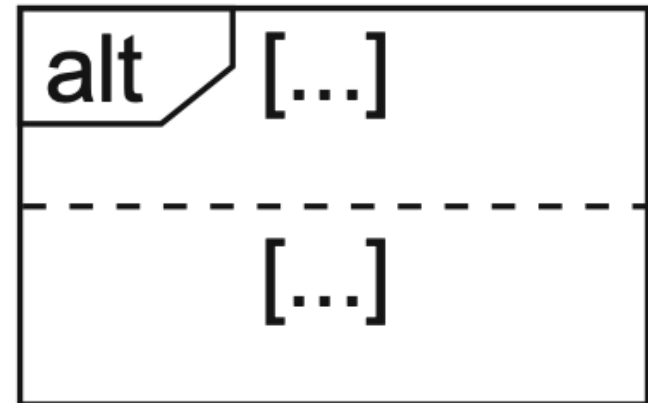
- Ví dụ
 - Một đối tượng nào đó thuộc lớp **Professor** muốn tạo một đối tượng thuộc lớp **ExamDate**



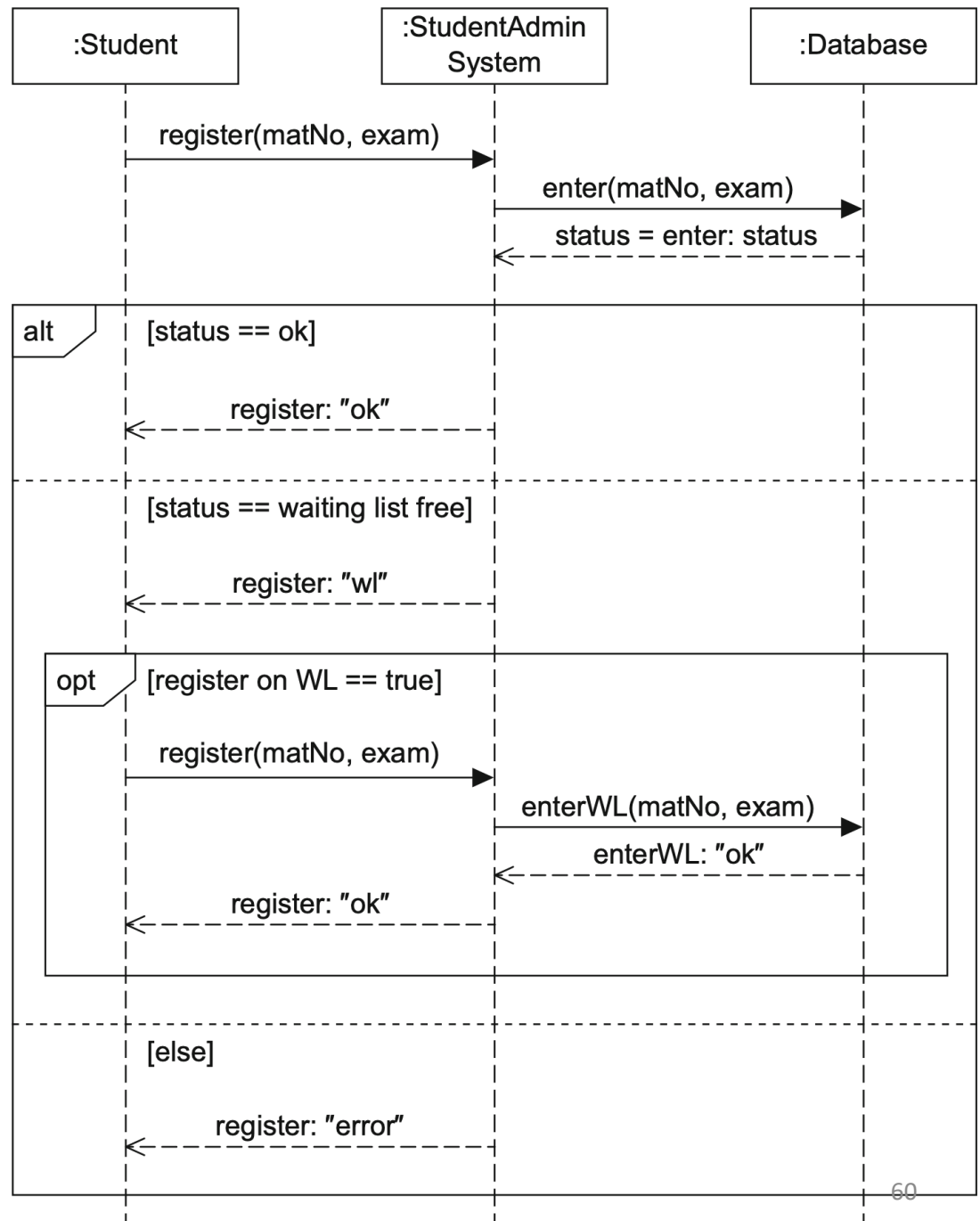
Khối điều khiển (alt)

- If else ...
- Tên khối alt
- Điều kiện thực hiện được đặt trong [...]

alt fragment



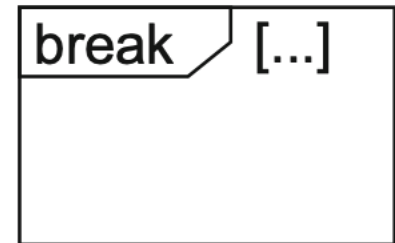
Ví dụ



Khối break

- Tương tự câu lệnh return
- Tên khối break
- Điều kiện thực hiện được đặt trong [...]

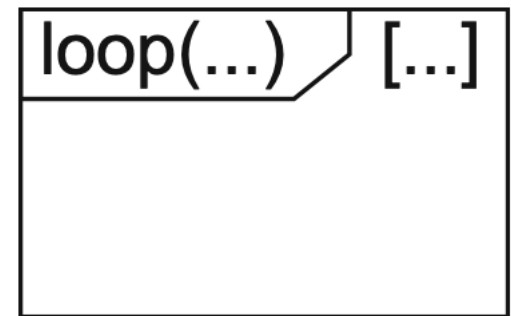
break fragment
Exception handling



Khối điều khiển lặp lại (loop)

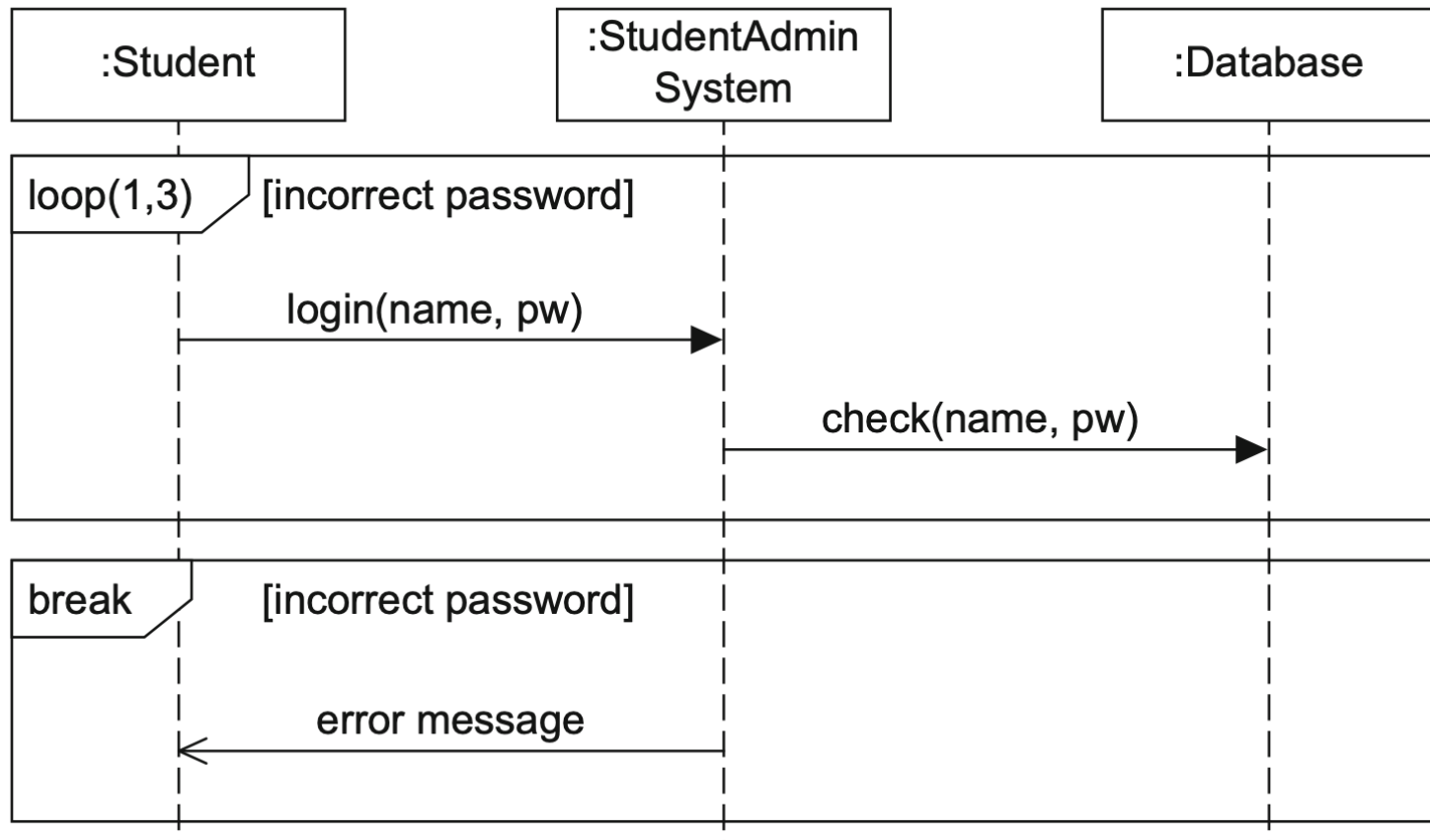
- while ... do
- Tên khối loop
- Điều kiện thực hiện được đặt trong [...]

loop fragment
Repeated interaction



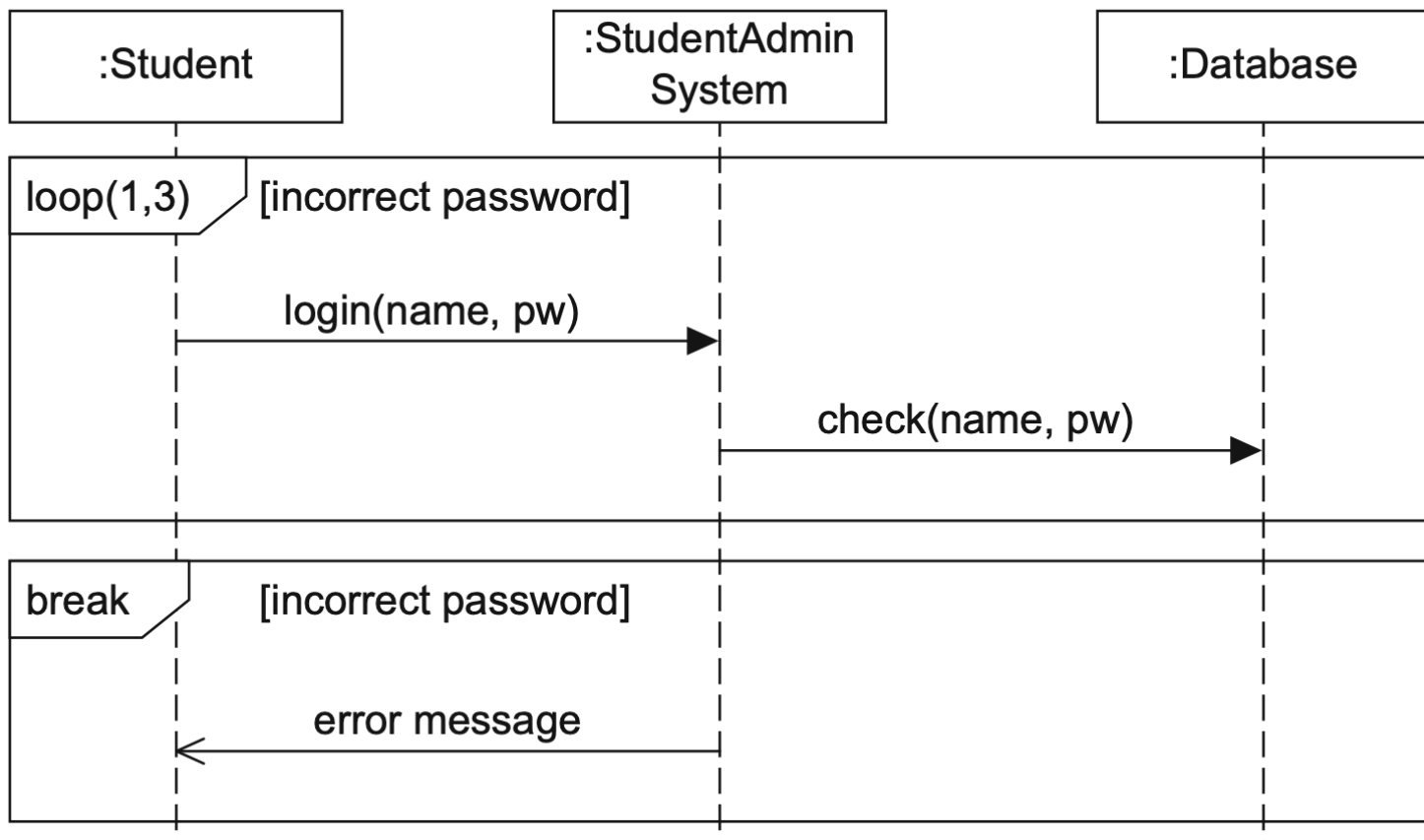
Khối điều khiển loop và break

- Giải thích hình sau



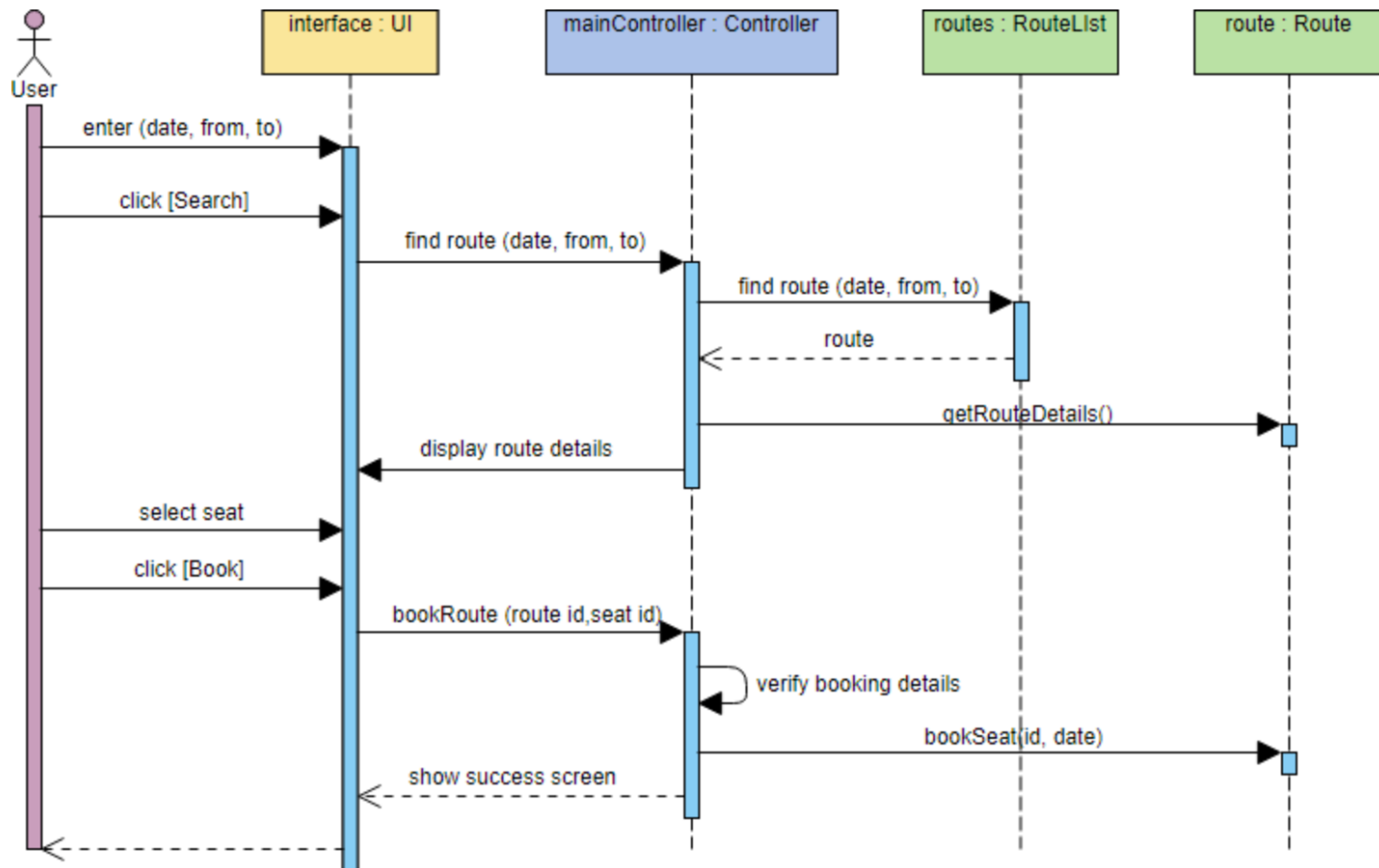
Khối điều khiển loop và break

- SV được nhập sai thông tin ba lần, nếu qua lần thì thông báo lỗi và kết thúc



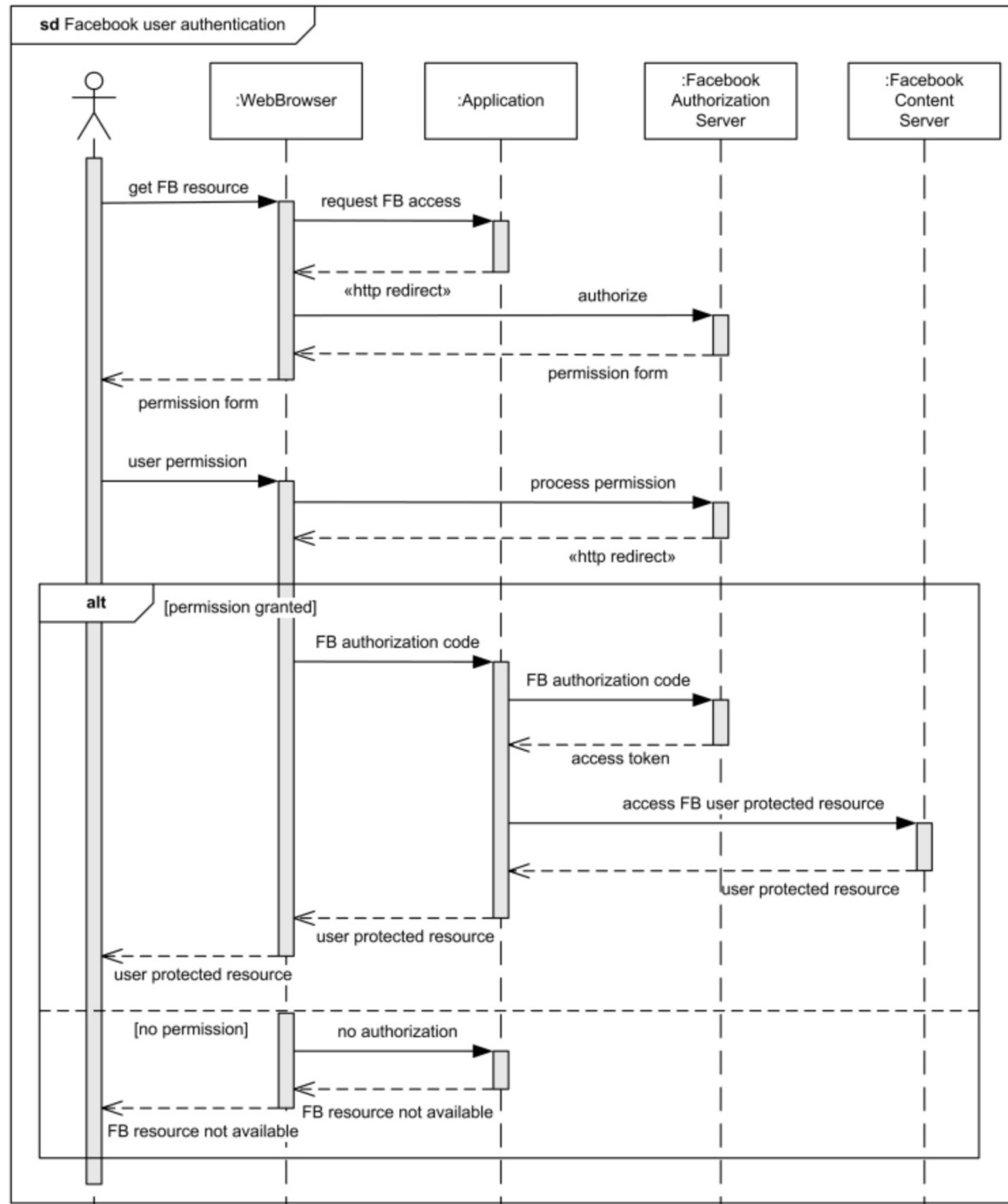
Giải thích hình sau

- Mô tả tương tác người dùng với hệ thống đặt vé



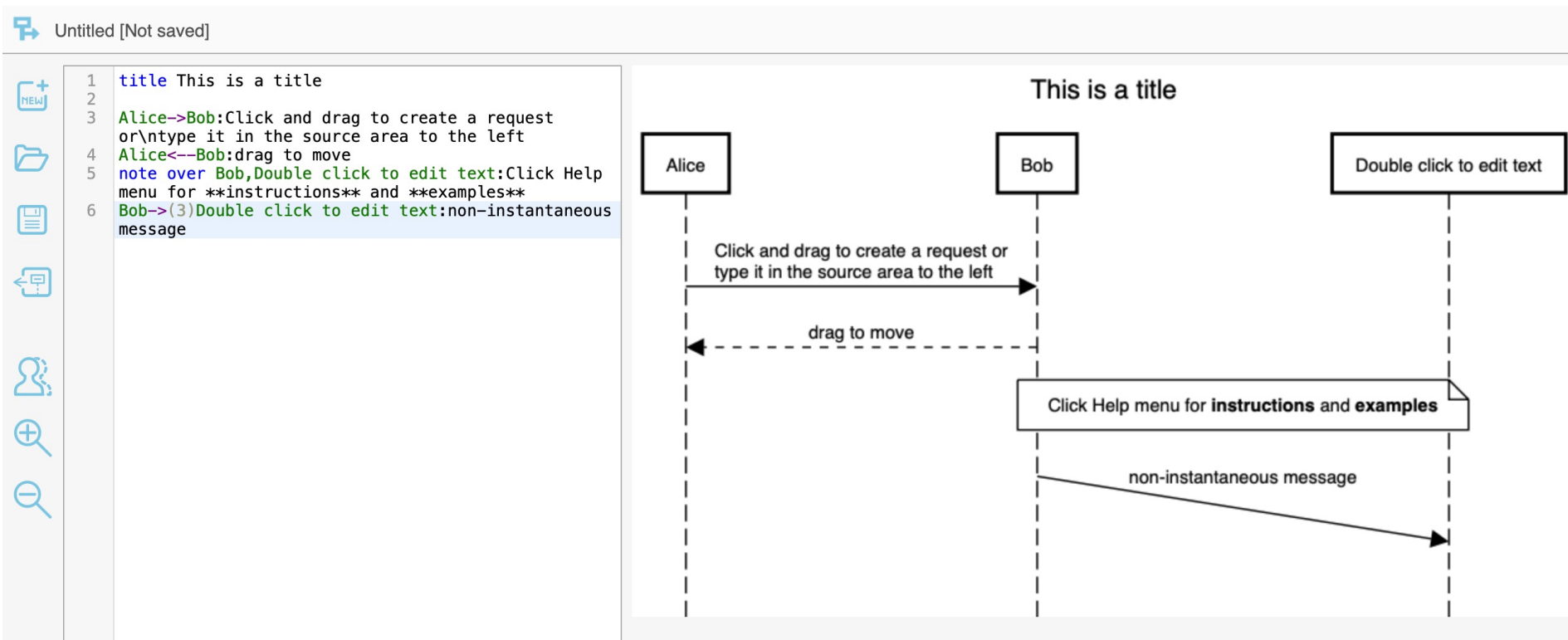
Giải thích hình sau

Quy trình xác thực người dùng Facebook



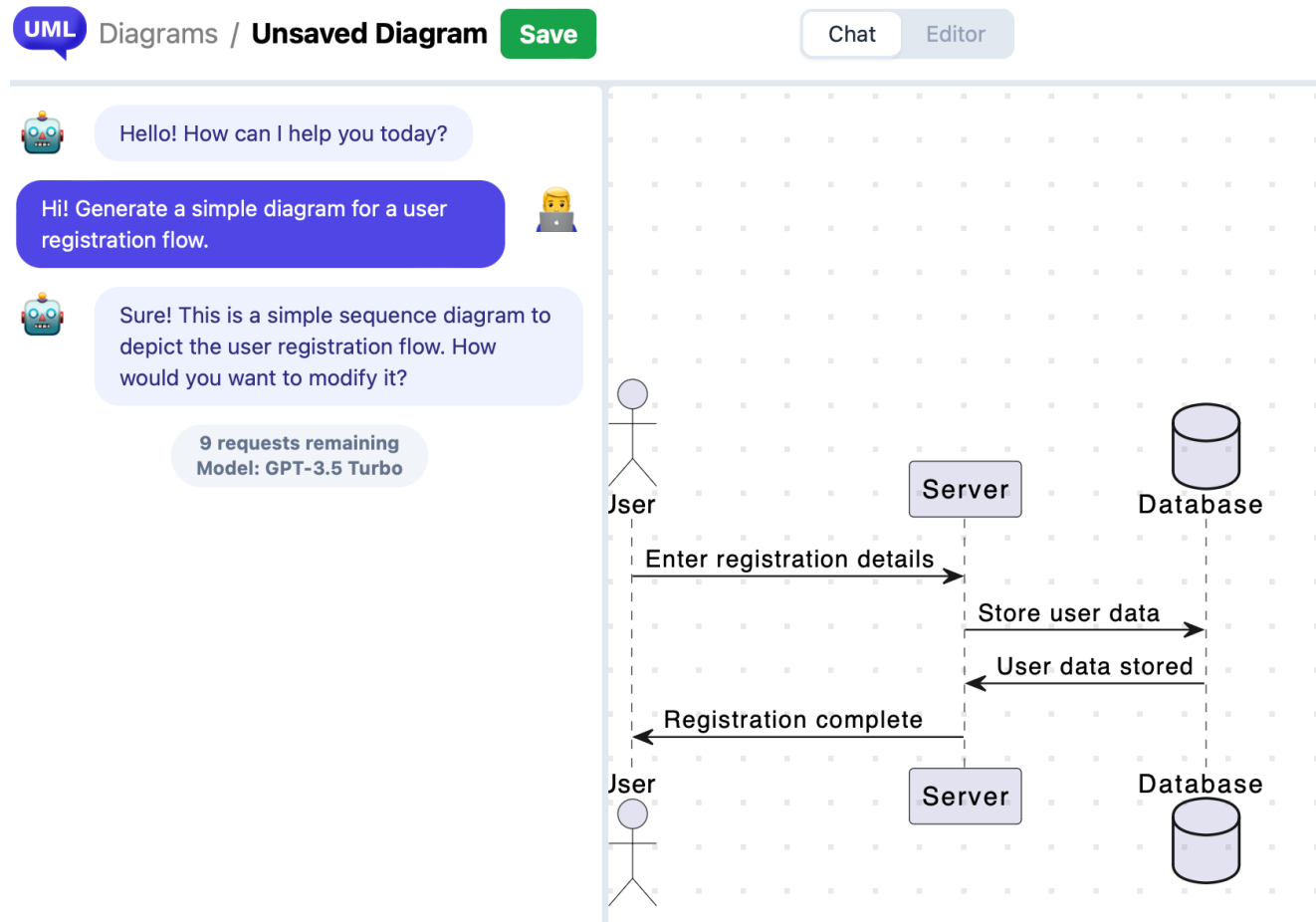
Tool

- Vẽ biểu đồ tuần tự online:
<https://sequencediagram.org>



Tool

- Hỏi AI để vẽ: <https://chatuml.com>



KỸ NGHỆ HƯỚNG MÔ HÌNH

Kỹ nghệ hướng mô hình

- Kỹ nghệ hướng mô hình là phương pháp phát triển phần mềm trong đó các mô hình được xem là các chế tác đầu ra cơ bản của quy trình phát triển (chương trình là một dạng mô hình đầu ra)
- Các chương trình tương ứng với nên tảng thực thi nào đó thường được sinh tự động từ các mô hình bởi các chuyển đổi mô hình.
- Hướng tiếp cận này cho phép các kỹ sư làm việc ở cấp độ trừu tượng cao => dễ giao tiếp với chuyên gia miền, giảm nỗ lực ánh xạ thiết kế sang các diễn đạt chi tiết ở cấp độ ngôn ngữ lập trình.

Đặc điểm kỹ nghệ hướng mô hình

- Ưu điểm
 - xem xét hệ thống ở cấp độ trừu tượng cao
 - sinh mã tự động
- Nhược điểm
 - Sai khác giữa các trừu tượng mức mô hình và mức cài đặt
 - chi phí để tạo ra các bộ chuyển đổi mô hình cũng như các bộ sinh mã tự động

Tổng kết

- Khái niệm mô hình hóa hệ thống
- Ngôn ngữ mô hình hóa UML
- Các góc nhìn hệ thống phần mềm
- Một số loại mô hình hệ thống
- Kỹ nghệ hướng mô hình